Universidad Autónoma de Chihuahua

Facultad de Ingeniería

Detección de la Abeja Reina en una Colmena por Medio del Aprendizaje Profundo

---

Análisis de Frecuencias Emitidas por el Panal

Student: Leonardo Rafael León Mora.

Thesis director: Dr. Daniel Espinobarro Velázquez.

Advisors:

- M.C. Carlos Hugo Larrinúa Pacheco.
- M.I. Joseph Isaac Ramírez Hernández.

Chihuahua, Chih., 2 de abril de 2026.

Índice general

RESUMEN **4**

1. INTRODUCCIÓN **5**
1.1. Planteamiento del Problema 5
1.2. Justificación 6
1.3. Objetivos 6
1.3.1. Objetivo General 6
1.3.2. Objetivos Específicos 6
1.4. Hipótesis de Trabajo 6

2. MARCO TEÓRICO **7**
2.1. Las Abejas Melíferas y su Importancia Ecológica y Económica 7
2.1.1. Rol de las Abejas en la Agricultura y Ecosistemas 7
2.1.2. Declive de Poblaciones y Factores de Riesgo 7
2.1.3. Estructura Social de la Colmena y Rol de la Reina 7
2.1.4. Necesidad de Sistemas de Monitoreo No Invasivos 8
2.2. Monitoreo Acústico de Colmenas: Estado del Arte 8
2.2.1. Fundamentos del Análisis Acústico en Apicultura 8
2.2.2. Comunicación Acústica en Abejas Melíferas 8
2.2.3. Detección de Eventos mediante Análisis Acústico 9
2.2.4. Otros Enfoques de Machine Learning 10
2.2.5. Limitaciones de Estudios Previos y Brechas de Conocimiento 11
2.3. Fundamentos de Procesamiento Digital de Señales de Audio 11
2.3.1. Representación Digital de Señales de Audio 11
2.3.2. Características de Señales de Audio de Colmenas 11
2.3.3. Transformada de Fourier de Tiempo Corto (STFT) 12
2.3.4. Transformada Rápida de Fourier (FFT) 13
2.4. Coeficientes Cepstrales en Frecuencia Mel (MFCC) 14
2.4.1. Motivación y Fundamento Perceptual 14
2.4.2. La Escala Mel 14
2.4.3. Proceso de Extracción de MFCC 14
2.4.4. Configuraciones de MFCC en Estudios de Colmenas 18

2.4.5. Implementación Práctica . . . 18
2.5. Aprendizaje Profundo para Clasificación de Audio . . . 19
2.5.1. Conceptos Fundamentales de Redes Neuronales . . . 19
2.5.2. Perceptrón Multicapa (MLP) . . . 20
2.5.3. Regresión Logística . . . 20
2.5.4. Redes Neuronales Recurrentes (RNN) y LSTM . . . 20
2.5.5. Redes Neuronales Convolucionales (CNN) . . . 22
2.5.6. Comparación de Arquitecturas . . . 24
2.6. Síntesis y Justificación Metodológica . . . 24
2.6.1. Contexto y Motivación . . . 24
2.6.2. Estado del Arte en Monitoreo Acústico . . . 24
2.6.3. Fundamentos Técnicos . . . 24
2.6.4. Justificación de la Metodología Propuesta . . . 25

3. METODOLOGÍA **26**
3.1. Procedimiento . . . 26
3.2. Arquitectura del Modelo . . . 30
3.3. Distribución del Conjunto de Datos . . . 31
3.4. Instrumentos . . . 32
3.5. Base de Datos . . . 32

4. RESULTADOS **33**
4.1. Resultados del Entrenamiento . . . 33
4.2. Evaluación en el Conjunto de Prueba . . . 33
4.3. Análisis Temporal de Predicciones . . . 34

5. DISCUSIÓN **36**
5.1. Interpretación de los Resultados . . . 36
5.1.1. Análisis del Primer Modelo (Figura 4.1) . . . 36
5.1.2. Análisis del Segundo Modelo (Figura 4.2) . . . 36
5.2. Comparaciones . . . 37
5.2.1. Comparación entre Modelos . . . 37
5.2.2. Comparación con el Estado del Arte . . . 37
5.3. Limitaciones y Desafíos Identificados . . . 37
5.3.1. Generalización . . . 37
5.3.2. Desbalance de Clases . . . 37
5.3.3. Robustez al Ruido . . . 38

5.3.4. Implicaciones Prácticas 38
5.3.5. Direcciones Futuras 38

6. CONCLUSIONES Y RECOMENDACIONES **39**
6.1. Conclusiones 39

BIBLIOGRAFÍA **40**

APÉNDICE A **42**
Programa # 1 42
Programa # 2 48
Programa # 3 53
Programa # 3 57

RESUMEN

1. INTRODUCCIÓN

Las abejas melíferas han sido ampliamente estudiadas por su destacada contribución al medio ambiente, proporcionando recursos que son vitales para animales, plantas y seres humanos. Su papel en la polinización es crucial para la biodiversidad y la producción agrícola[1]. Además, la miel, el polen, la jalea real y la cera de abejas son productos altamente valorados por sus propiedades nutricionales y medicinales[2].

Durante las temporadas de mayor producción de miel, como en primavera y verano, las abejas melíferas pueden recorrer distancias de hasta 6 km[3], viajando aproximadamente 3 km de ida y vuelta entre la colmena y las fuentes de alimento, como flores, para recolectar polen, néctar o agua. Pero, ¿por qué surge este comportamiento? ¿Cuál es la motivación de las abejas para regresar a la colmena y abastecerla?

La comunicación dentro de la colmena es esencial para la supervivencia de la colonia. Las abejas utilizan una variedad de señales, incluyendo feromonas y sonidos, para coordinar sus actividades y mantener el orden social[2]. La presencia de una abeja reina es fundamental para la cohesión del grupo, ya que su ausencia puede conducir a un comportamiento caótico y al eventual colapso de la colonia[4]. Otra forma de comunicación es el *waggle dance*, mediante el cual las abejas pueden informar sobre la ubicación de fuentes de alimento, tomando como referencia los puntos cardinales y trazando un vector en la dirección requerida[5], lo que permite notificar a las demás abejas recolectoras.

Según Ruvinga[4], "*La colonia está formada por la reina, miles de abejas obreras estériles, unos pocos cientos de zánganos, huevos, larvas y pupas*". Las abejas trabajan a diario, realizando tareas que se dividen no solo entre machos y hembras, sino también entre abejas jóvenes y adultas[6]. Un panal puede albergar entre 30,000 y 50,000 abejas, aunque este número puede reducirse hasta 5,000 durante el invierno. Este comportamiento de abastecimiento constante está estrechamente vinculado a la estructura social de la colmena y, en particular, al papel de la abeja reina, que es la única capaz de procrear[4].

1.1. Planteamiento del Problema

La reina puede poner en promedio hasta 2,000 huevos al día. Las obreras se encargan de construir, defender y abastecer la colmena con recursos esenciales como agua, polen y néctar[2]. Los zánganos cumplen funciones reproductivas y también se cree que ayudan a regular la temperatura del panal, mientras que las abejas más jóvenes se limitan a tareas dentro de la colmena[6]. La pérdida de la abeja reina puede comprometer seriamente la supervivencia de la colonia.

Las colmenas están expuestas a múltiples amenazas: insectos invasores, pesticidas, condiciones climáticas e inclusive seres humanos. En sistemas apícolas tradicionales donde se utilizan cajas de madera, cada inspección física implica abrir la colmena y romper el sello de propóleo que las abejas han construido para mantener un ambiente limpio y controlado. Esta intervención frecuente no solo representa una pérdida energética para la colonia, que debe reconstruir el sello de propóleo y reabastecer el panal con respecto a las abejas que mueren en este proceso; sino que además incrementa el riesgo de aplastar accidentalmente a la reina durante la manipulación interna de la colmena, lo que puede conducir al colapso de toda la colonia si no se toma acción inmediata.

Una abeja reina puede ser reemplazada por la colonia mediante la crianza de una nueva

reina a partir de larvas seleccionadas, siempre y cuando existan suficientes recursos y tiempo para que la nueva reina se desarrolle completamente. Este proceso puede tardar entre 16 y 24 días, dependiendo de las condiciones ambientales y la disponibilidad de alimento[2]. Durante este período, la colonia puede volverse vulnerable a enfermedades, depredadores y otros factores estresantes que pueden afectar su supervivencia. Como una alternativa, los apicultores pueden introducir una reina comprada o criada en otro lugar para restablecer la estabilidad de la colonia. Sin embargo, la aceptación de una nueva reina por parte de las obreras no está garantizada y puede variar según el comportamiento de la colonia y las condiciones ambientales[4].

1.2. Justificación

Contar con herramientas que permitan identificar la presencia o ausencia de la abeja reina de manera no invasiva resulta de gran utilidad para garantizar la continuidad y productividad de la colmena. Esta investigación propone un enfoque basado en el análisis acústico del panal, aprovechando técnicas de aprendizaje automático para detectar el estado de la reina. Dado que estudios recientes han demostrado que el sonido dentro de la colmena varía según la presencia o ausencia de la reina[4, 7, 8, 9], se busca validar un sistema que, a partir de grabaciones de audio, identifique estas variaciones de forma automática y eficiente.

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar un sistema de detección automática con respecto a la presencia de la abeja reina en una colmena, a partir del cual se permita la recolección de datos, el análisis de audio y la implementación de técnicas de aprendizaje automático. Se pretende obtener un modelo capaz de clasificar el estado de la reina en función de las características acústicas extraídas de grabaciones de audio. Este enfoque no invasivo busca minimizar la perturbación de la colonia durante las inspecciones recurrentes que suele hacer un apicultor, preservando así el bienestar de las abejas y la integridad del panal.

1.3.2. Objetivos Específicos

- ☞ Realizar una revisión bibliográfica sobre la estructura de las colonias de abejas melíferas y los efectos de la ausencia de la reina.
- ☞ Procesar y organizar un conjunto de datos de audio de colmenas etiquetado con el estado de la reina.
- ☞ Extraer características acústicas relevantes, como los coeficientes cepstrales en la escala Mel (MFCC), a partir de las grabaciones.
- ☞ Entrenar y evaluar modelos de aprendizaje automático para clasificar el estado de la reina basado en características extraídas.
- ☞ Validar la efectividad del sistema en términos de precisión, sensibilidad y especificidad.

1.4. Hipótesis de Trabajo

Dado que el comportamiento sonoro de una colmena varía según el estado de su reina, se espera que un sistema basado en el análisis de características acústicas sea capaz de detectar, con un grado aceptable de precisión, si la reina está presente, ausente, rechazada o recién aceptada en la colonia.

2. MARCO TEÓRICO

2.1. Las Abejas Melíferas y su Importancia Ecológica y Económica

2.1.1. Rol de las Abejas en la Agricultura y Ecosistemas

Las abejas melíferas (*Apis mellifera*) constituyen uno de los grupos de polinizadores más importantes a nivel mundial, visitando más del 90 % de los 107 principales tipos de cultivos globales [10]. Su labor de polinización contribuye directamente a la producción de aproximadamente el 75 % de los cultivos de importancia alimentaria, incluyendo la mayoría de frutas, semillas y nueces, así como cultivos de alto valor comercial como café, cacao y canola [11].

El valor económico de los servicios de polinización animal se estima entre USD $235-577 mil millones anuales a nivel global [11], representando aproximadamente el 9.5 % del valor total de la producción agrícola destinada al consumo humano. Sin embargo, más allá del valor monetario directo, la polinización por abejas es fundamental para la seguridad alimentaria global: se estima que entre 5-8 % de la producción mundial de cultivos se perdería sin servicios de polinización animal [10], lo que requeriría cambios significativos en las dietas humanas y una expansión desproporcionada de tierras agrícolas.

2.1.2. Declive de Poblaciones y Factores de Riesgo

En las últimas décadas se ha documentado un declive alarmante en las poblaciones de abejas melíferas en múltiples regiones del mundo, particularmente en Europa occidental y Norteamérica [11]. Este declive ha sido atribuido a diversos factores interrelacionados, incluyendo:

- **Cambios en el uso del suelo**: La conversión de hábitats semi-naturales a agricultura intensiva reduce la disponibilidad de recursos florales y sitios de anidación [11].
- **Pesticidas**: Particularmente los neonicotinoides, que han demostrado efectos letales y subletales en polinizadores, afectando su navegación, aprendizaje y reproducción [11].
- **Patógenos y parásitos**: El ácaro *Varroa* y virus asociados representan amenazas significativas para la salud de las colonias [11].
- **Cambio climático**: Alteraciones en la sincronización temporal y espacial entre las abejas y las plantas que polinizan [11].
- **Especies invasoras**: Introducción de depredadores y competidores que alteran las redes de polinización establecidas.

2.1.3. Estructura Social de la Colmena y Rol de la Reina

Las abejas melíferas son insectos eusociales altamente organizados. Una colonia típica en verano puede contener entre 30,000 y 80,000 individuos, reduciendo a tan solo 5,000 en invierno. La colmena está compuesta por tres castas principales: la reina (única hembra fértil), miles de obreras (hembras estériles) y cientos de zánganos (machos) durante la temporada reproductiva [4].

La reina es el eje central de la colonia, siendo la única capaz de poner huevos fertilizados.

Su presencia es crítica para la supervivencia a largo plazo de la colmena, ya que regula la cohesión social mediante la producción de feromonas específicas, particularmente la feromona mandibular de la reina (QMP) [9]. **La pérdida de la reina**, conocida como estado de *queenlessness*, resulta en un declive gradual de la población que **eventualmente lleva a la extinción de la colonia** si no se toman medidas correctivas oportunas [9].

Las obreras realizan todas las tareas necesarias para el mantenimiento de la colonia, incluyendo construcción de panales, recolección de néctar y polen (forrajeo), alimentación de larvas, regulación de temperatura, defensa y limpieza. Sus funciones varían según su edad en un proceso conocido como *polietismo temporal* [4].

Los zánganos tienen como función primordial la fecundación de reinas vírgenes. Aunque no participan en tareas laborales dentro de la colmena, pueden contribuir a la termorregulación del nido [4].

2.1.4. Necesidad de Sistemas de Monitoreo No Invasivos

El monitoreo tradicional de colmenas mediante inspección visual directa es laborioso, requiere experiencia, puede ser estresante para las abejas y potencialmente disemina enfermedades y parásitos entre colonias. Además, existe el riesgo de muerte accidental de la reina durante la inspección [9]. Por estas razones, existe una demanda creciente por sistemas de monitoreo remoto, no invasivo y continuo que permitan a los apicultores detectar tempranamente problemas en sus colmenas.

2.2. Monitoreo Acústico de Colmenas: Estado del Arte

2.2.1. Fundamentos del Análisis Acústico en Apicultura

Las abejas producen una variedad de señales acústicas y vibratorias que transmiten información sobre el estado de la colonia. Estas señales son generadas principalmente por el batido de alas, aunque también incluyen vibraciones torácicas transmitidas al sustrato [8]. El análisis de estas señales acústicas ofrece una modalidad de monitoreo apropiada porque:

- Es completamente no invasivo.
- Permite el monitoreo continuo y en tiempo real.
- Requiere equipamiento relativamente simple (micrófono).
- Proporciona información rica sobre eventos importantes en la colmena.

2.2.2. Comunicación Acústica en Abejas Melíferas

Estudios pioneros han documentado diversos tipos de señales acústicas producidas por las abejas [8]. La Tabla 2.1 resume las principales señales identificadas:

Cuadro 2.1: **Señales acústicas y vibratorias en *Apis mellifera***

Señal	Frecuencia	Duración	Contexto/Función
Queen Piping	340-450 Hz	1.25 s	Producida por reina virgen recién emergida
Queen Quacking	< 450 Hz	< 200 ms	Respuesta de reinas no emergidas
Worker Piping (alas juntas)	200-2000 Hz	0.2-2 s	En enjambres, modulación en frecuencia
Worker Piping (alas separadas)	300-400 Hz	0.2-2 s	En colmenas, sin modulación
Danza del meneo	200-300 Hz + 13-15 Hz (cola)	Pulsos cortos	Comunicación de ubicación de recursos
Señal de detención	350 Hz (media)	0.05-0.2 s	Interrupción de reclutamiento
Señal de vibración	16-18 Hz	1-2 s	Activación general de obreras

2.2.3. Detección de Eventos mediante Análisis Acústico

Detección de un comportamiento de enjambre (*Swarming*)

Swarming es el proceso natural de reproducción de colonias mediante el cual una porción significativa de la colonia abandona la colmena original con la reina vieja para establecer una nueva colonia. Este evento representa una pérdida económica considerable para el apicultor.

Ferrari et al. (2008)[12] realizaron uno de los primeros estudios cuantitativos sobre la predicción del Swarming mediante análisis acústico. Monitorearon sonidos y temperatura en el periodo alrededor de un evento de Swarming, observando que las bandas de frecuencia dominantes se desplazan de 100-300 Hz en condiciones normales a 500-600 Hz inmediatamente antes del enjambre. Este incremento fue atribuido al batido intensivo de alas en preparación para el vuelo.

Bencsik et al. (2011) utilizaron acelerómetros en lugar de micrófonos, capturando vibraciones del panal. Mediante análisis de componentes principales (PCA) y eigenespectros identificaron patrones característicos previos a Swarming.

Detección de Ausencia de Reina (*Queenlessness*)

La detección del estado *queenless* es crítica porque permite intervención temprana antes de que la colonia decline irreversiblemente.

Boys (1999) reportó observaciones empíricas de apicultores experimentados, notando que el sonido "warble" característico entre 225-285 Hz disminuye en amplitud en colmenas sin reina, siendo reemplazado por un sonido "quejumbroso" a frecuencias ligeramente menores. Adicionalmente, el sonido general fue descrito como discordante en colmenas *queenless*, en contraste con el sonido armonioso de colmenas *queenright* (aquellos panales que si conservan a la reina).

Howard et al. (2013) [8] realizaron el primer estudio cuantitativo sistemático sobre detección de *queenlessness* mediante procesa-

miento de señales. Utilizaron:

- Grabaciones de cuatro colmenas (dos control *queenright*, dos experimentales con reina removida).
- Análisis espectral mediante Transformada de Stockwell.
- Clasificación mediante Mapas Autoorganizados (SOM).
- Análisis de Densidad Espectral de Potencia (PSD).

Aunque sus resultados fueron algo inconclusos, establecieron la metodología base y confirmaron diferencias espectrales entre ambos estados, particularmente en el rango 165-285 Hz.

Ruvinga et al. (2021, 2023) representan el avance más significativo en esta área, aplicando técnicas modernas de aprendizaje profundo. Su trabajo se desarrolló en dos fases:
Fase 1 (2021)[4]**:** Comparación de clasificadores usando MFCC

- **Dataset**: 480 grabaciones de 1 minuto (Aug 2012, 4 colmenas).
- **Características**: 14 MFCC (13 coeficientes + log-energía).
- **Modelos evaluados**: Regresión Logística, MLP, LSTM.
- **Resultados**: LSTM alcanzó 92 % de precisión, superando a MLP (90 %) y Regresión Logística (87 %).

Fase 2 (2023)[9]**:** Extensión con CNN y validación cruzada

- Incorporación de espectrogramas STFT como entrada para CNN.
- CNN alcanzó 99.31 % de precisión (training) y 99 % (validación).
- Validación en dataset independiente (Surrey, UK, verano 2020): CNN mantuvo 98.61 % de precisión.
- Confirmación de generalización del modelo a diferentes ubicaciones y temporadas.

La Tabla 2.2 resume los resultados comparativos:

Cuadro 2.2: **Comparación de modelos para detección de *queenlessness***

Modelo	Características	Precisión	Recall	F1-Score
Reg. Logística	14 MFCC	87.4 %	-	-
MLP	14 MFCC	90.1 %	-	-
LSTM	14 MFCC	92.0 %	92 %	92 %
CNN	**Espectrograma**	**99.31 %**	**99.31 %**	**99.31 %**

2.2.4. Otros Enfoques de Machine Learning

Robles-Guerrero et al. (2019, 2020)[13] exploraron diferentes aproximaciones:

- **Lasso Logistic Regression** combinada con Singular Value Decomposition (SVD) para clasificación multiclase de estados de colmena.
- Identificación de patrones sonoros específicos en colmenas sin reina.
- Enfoque en características espectrales de alta dimensión reducidas mediante SVD.

Peng et al. (2020)[14] reportaron el uso de MLP con dos capas ocultas y función de activación tangente hiperbólica, alcanzando 90 % de precisión con señales limpias, aunque el desempeño cayó a 61 % con señales ruidosas, evidenciando la importancia del preprocesamiento de señales.

2.2.5. Limitaciones de Estudios Previos y Brechas de Conocimiento

A pesar de los avances significativos, persisten limitaciones importantes:

1. **Datos limitados**: la mayoría de los estudios usan datasets pequeños de pocos días y pocas colmenas.
2. **Condiciones controladas**: pocos estudios se validan en condiciones de campo reales con ruido ambiental variable.
3. **Generalización**: falta evidencia de que modelos funcionen en diferentes:
   - Razas de abejas (*A. m. ligustica* vs *A. m. carnica*, etc.).
   - Condiciones climáticas y estaciones del año.
   - Ubicaciones geográficas.
4. **Sistemas embebidos**: pocos trabajos abordan la implementación en hardware de bajo costo para monitoreo en tiempo real.

2.3. Fundamentos de Procesamiento Digital de Señales de Audio

2.3.1. Representación Digital de Señales de Audio

Una señal de audio continua $x(t)$ se convierte en una señal digital mediante muestreo a intervalos regulares determinados por la frecuencia de muestreo f_s. La señal digital resultante se representa como una secuencia discreta:

$$\begin{aligned} x[n] &= x(t = nT) \ , \\ n &= 0, 1, 2, \ldots, N-1 \ , \end{aligned} \tag{2.1}$$

donde $T = 1/f_s$ siendo este término el periodo de muestreo y N es el número total de muestras.

Según el **Teorema de Muestreo de Nyquist-Shannon**, para evitar aliasing (solapamiento espectral), la frecuencia de muestreo debe ser al menos el doble de la frecuencia máxima contenida en la señal:

$$f_s \geq 2 f_{\max} \ . \tag{2.2}$$

En estudios de audio de colmenas, Ruvinga et al. [9] utilizaron $f_s = 44.1$ kHz, mientras que otros trabajos han usado 20.38 kHz, ambos valores adecuados considerando que la energía significativa en sonidos de abejas se concentra por debajo de 11 kHz [9].

2.3.2. Características de Señales de Audio de Colmenas

Las señales de audio de colmenas presentan características particulares [9]:

- **No estacionarias**: sus propiedades estadísticas varían en el tiempo.
- **Multicomponente**: contienen componentes armónicos (batido de alas), transitorios (impactos, movimientos) y estocásticos (ruido de fondo).

- **Energía concentrada**: frecuencia fundamental ∼100 Hz, con información relevante hasta 6 kHz.
- **Variabilidad temporal**: cambios según hora del día, actividad de forrajeo y condiciones climáticas.

2.3.3. Transformada de Fourier de Tiempo Corto (STFT)

Debido a la naturaleza no estacionaria de las señales de audio, el análisis frecuencial tradicional mediante Transformada de Fourier (DFT) es insuficiente, ya que proporciona el contenido frecuencial promedio pero pierde completamente la información temporal.

La **Transformada de Fourier de Tiempo Corto (STFT)** resuelve esta limitación dividiendo la señal en segmentos cortos (ventanas) donde se puede asumir estacionariedad local, aplicando la DFT a cada segmento:

$$X[m,k] = \sum_{n=0}^{L-1} x[n+mH] \cdot w[n] \cdot e^{-j\frac{2\pi kn}{N}} \ , \quad (2.3)$$

donde:

- $x[n]$: señal de entrada.
- $w[n]$: función de ventana de longitud L.
- m: índice de trama ($m = 0, 1, 2, \ldots$).
- H: hop size (salto entre ventanas, típicamente $H = L/2$ para 50 % de traslape).
- k: índice de frecuencia ($k = 0, 1, \ldots, N-1$).
- N: longitud de la FFT (típicamente $N \geq L$, potencia de 2 para eficiencia).

El **espectrograma** se obtiene como el módulo al cuadrado de la STFT:

$$S[m,k] = |X[m,k]|^2 \ , \quad (2.4)$$

el cual representa la densidad de energía en el plano tiempo-frecuencia y se visualiza típicamente como una imagen donde:

- Eje horizontal: tiempo (índice de trama m).
- Eje vertical: frecuencia (índice de bin k).
- Intensidad/color: magnitud de la energía.

Segmentación y Ventaneo

La señal se divide en **tramas** (*frames*) de duración típica entre 20-40 ms, con un traslape (*overlap*) entre tramas consecutivas. Esta duración se selecciona porque:

- Es suficientemente corta para capturar variaciones rápidas.
- Es suficientemente larga para obtener resolución frecuencial adecuada.
- Minimiza efectos de no-estacionariedad local.

Para una frecuencia de muestreo f_s y duración de trama D (en segundos), el número de muestras por ventana es:

$$L = \lfloor D \cdot f_s \rfloor \ .$$

Ejemplo: Para $f_s = 44100$ Hz y $D = 25$ ms:

$$L = \lfloor 0{,}025 \times 44100 \rfloor = 1102 \ .$$

En la práctica, se suele ajustar L a la potencia de 2 más cercana para eficiencia de la FFT, por ejemplo $L = 1024$.

El **hop size** H determina el desplazamiento entre tramas consecutivas. Para un traslape del 50 %:

$$H = \frac{L}{2} \ .$$

Esto significa que cada trama comparte la mitad de sus muestras con las tramas adyacentes. El traslape es particularmente importante

cuando se usan ventanas que atenúan los extremos, garantizando que cada muestra contribuya aproximadamente igual al análisis global.

El número total de tramas M para una señal de longitud N_{total} es:

$$M = \left\lfloor \frac{N_{\text{total}} - L}{H} \right\rfloor + 1 \ .$$

Funciones de Ventana

Se aplica una función de ventana $w[n]$ a cada trama para reducir el efecto de Gibbs (oscilaciones espectrales espurias) causado por la discontinuidad en los bordes de segmentos rectangulares. Las ventanas más comunes son:

Ventana de Hamming:

$$w[n] = 0{,}53836 - 0{,}46164 \cos\left(\frac{2\pi n}{L-1}\right) \ ,$$
$$0 \leq n \leq L-1 \ .$$

Frecuentemente aproximada como:

$$w[n] \approx 0{,}54 - 0{,}46 \cos\left(\frac{2\pi n}{L-1}\right) \ ,$$
$$0 \leq n \leq L-1 \ .$$

Ventana de Hanning (Hann):

$$w[n] = 0{,}5 - 0{,}5 \cos\left(\frac{2\pi n}{L-1}\right) \ ,$$
$$0 \leq n \leq L-1 \ .$$

Ambas se pueden expresar en forma generalizada como:

$$w[n] = (1-\omega) - \omega \cos\left(\frac{2\pi n}{L-1}\right) \ ,$$
$$0 \leq n \leq L-1 \ .$$

donde $\omega = 0{,}5$ para Hanning y $\omega \approx 0{,}46$ para Hamming.

La ventana de Hamming ofrece mejor supresión de lóbulos laterales (∼43 dB) comparada con Hanning (∼31 dB), lo que reduce interferencia espectral entre componentes frecuenciales adyacentes.

2.3.4. Transformada Rápida de Fourier (FFT)

La FFT es un algoritmo eficiente para calcular la Transformada Discreta de Fourier (DFT). Para una trama ventaneada $x_w[n] = x[n] \cdot w[n]$:

$$X[k] = \sum_{n=0}^{N-1} x_w[n] \cdot e^{-j\frac{2\pi kn}{N}} \ ,$$
$$k = 0, 1, \ldots, N-1 \ .$$

La complejidad computacional de la FFT es $O(N \log N)$ comparada con $O(N^2)$ de la DFT directa, lo que la hace práctica para procesamiento en tiempo real.

Los bins frecuenciales resultantes tienen una resolución frecuencial de:

$$\Delta f = \frac{f_s}{N} \ .$$

Y cada bin k corresponde a una frecuencia física de:

$$f_k = \frac{k \cdot f_s}{N} \ ,$$
$$k = 0, 1, \ldots, \left\lfloor \frac{N}{2} \right\rfloor \ .$$

El rango útil de frecuencias (espectro unilateral) va de 0 Hz hasta la frecuencia de Nyquist $f_s/2$.

Ejemplo: Con $f_s = 44100$ Hz y $N = 1024$:

$$\Delta f = \frac{44100}{1024} \approx 43{,}07 \text{ Hz} \ ,$$

proporcionando bins frecuenciales de aproximadamente 43 Hz de ancho, con el espectro útil abarcando de 0 Hz a 22050 Hz (frecuencia de Nyquist).

2.4. Coeficientes Cepstrales en Frecuencia Mel (MFCC)

2.4.1. Motivación y Fundamento Perceptual

Los **Coeficientes Cepstrales en Frecuencia Mel (MFCC)** constituyen una representación compacta y robusta del contenido espectral de señales de audio, inspirada en el sistema auditivo humano [9]. Su diseño se basa en dos observaciones fundamentales de la psicoacústica:

1. **Percepción no lineal de frecuencias**: el oído humano no percibe frecuencias de manera lineal. Es más sensible a cambios en bajas frecuencias que en altas.
2. **Enmascaramiento frecuencial**: componentes espectrales cercanos se enmascaran mutuamente, haciendo innecesaria una resolución espectral uniforme muy alta.

Los MFCC han demostrado excelencia en múltiples dominios:

- Reconocimiento automático de voz (ASR) [9].
- Clasificación de sonidos ambientales.
- Clasificación de géneros musicales.
- Reconocimiento de sonidos cardíacos.
- **Clasificación de estados de colmenas** [4, 9].

2.4.2. La Escala Mel

La **escala Mel** es una escala perceptual de tonos (*pitch*) propuesta por Stevens et al. [15], definida de tal forma que intervalos iguales en la escala Mel son percibidos como igualmente espaciados por oyentes humanos.

La relación entre frecuencia física f (Hz) y frecuencia percibida m (Mel) se define como [16, 17]:

$$m = 2595 \log_{10} \left(1 + \frac{f}{700}\right) . \tag{2.5}$$

Alternativamente, con base natural:

$$m = 1127 \ln \left(1 + \frac{f}{700}\right) . \tag{2.6}$$

La transformación inversa es:

$$f = 700 \left(10^{m/2595} - 1\right) . \tag{2.7}$$

Por diseño, 1000 Hz corresponde a aproximadamente 1000 Mel, sirviendo como punto de referencia. La escala comprime progresivamente las frecuencias altas, modelando la mayor sensibilidad del oído humano a diferencias de tono en el rango de bajas frecuencias.

Esta relación aproxima el comportamiento del sistema auditivo humano:

- Alrededor de 1000 Hz: punto de referencia donde $m \approx f$ (1000 Hz $\approx$ 1000 Mel).
- Para frecuencias bajas: la percepción del tono es aproximadamente lineal con la frecuencia.
- Para frecuencias altas: la percepción del tono sigue una relación logarítmica.

La Figura 2.1 ilustra esta relación no lineal.

2.4.3. Proceso de Extracción de MFCC

La extracción de MFCC es un proceso de 7 etapas secuenciales [9], ilustrado en la Figura 2.2.

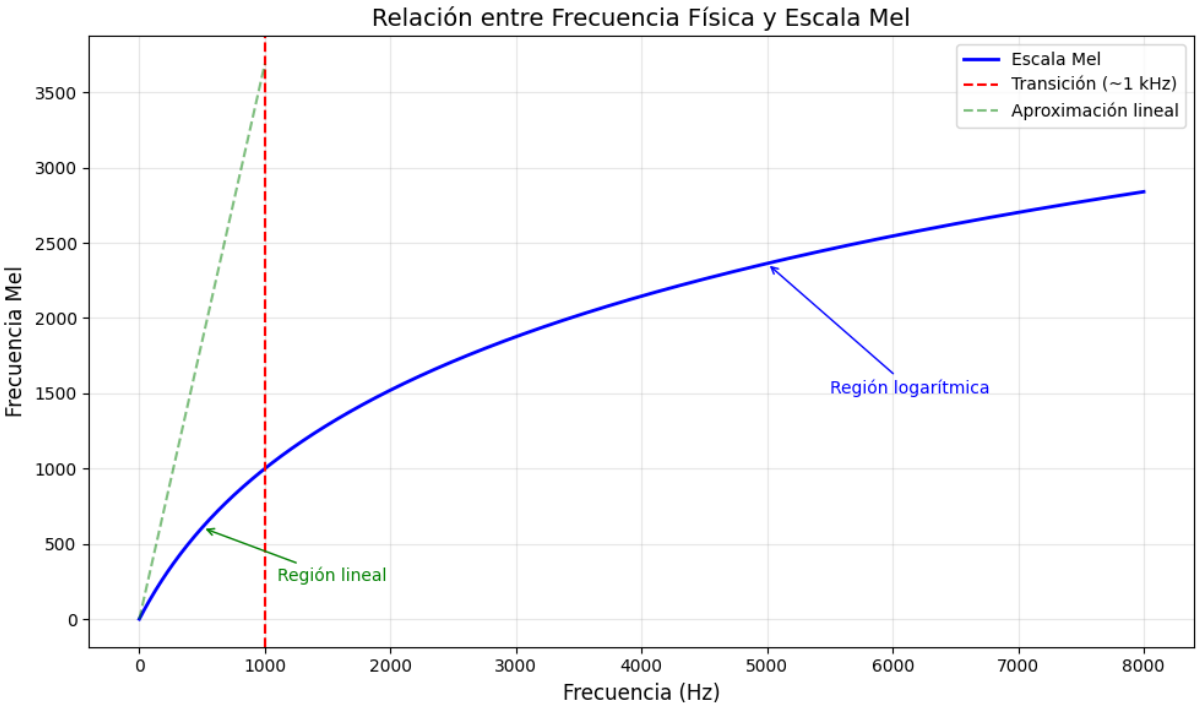


Figura 2.1: Relación entre frecuencia física (Hz) y escala Mel. Nótese la transición de comportamiento lineal a logarítmico alrededor de 1 kHz.

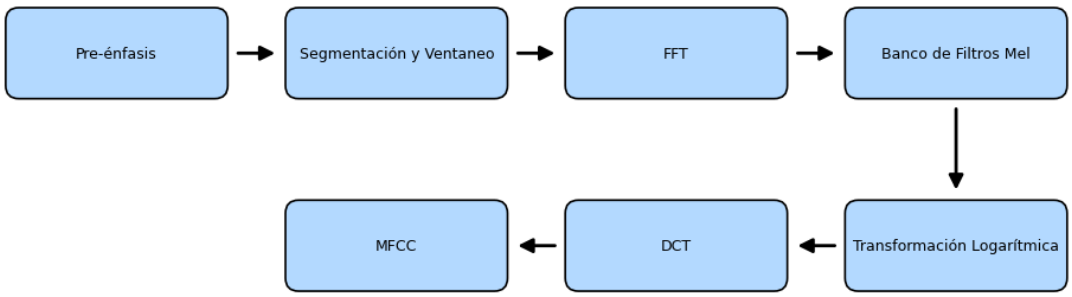


Figura 2.2: Proceso de extracción de MFCC en 7 etapas.

Etapa 1: Pre-énfasis

El **filtro de pre-énfasis** es un filtro pasa-altos de primer orden que amplifica las componentes de alta frecuencia, compensando la caída natural de energía en altas frecuencias que ocurre durante la generación y grabación de audio.

Se implementa como un filtro FIR de primer orden:

$$y[n] = x[n] - \alpha \cdot x[n-1] \ , \tag{2.8}$$

donde α es el coeficiente de pre-énfasis, típicamente $0{,}95 \leq \alpha \leq 0{,}97$.

En el dominio z:

$$H_{pre}(z) = 1 - \alpha z^{-1} \ . \tag{2.9}$$

En estudios de audio de colmenas, este filtro es particularmente importante porque [9]:

- Información relevante existe en bandas de 3-6 kHz.
- Estas frecuencias tienden a atenuarse durante grabación.
- El pre-énfasis mejora la discriminación espectral.

Etapa 2: Segmentación (*Framing*) y Ventaneo

Ya descrita en la sección 2.3.3, la señal se segmenta en tramas con traslape. Para MFCC, los parámetros típicos son [9]:

- Duración de trama: 20-40 ms (25 ms es común).
- Traslape: 50 % (12.5 ms si trama = 25 ms).
- Ventana: Hamming o Hanning.

Para cada trama i:

$$\begin{aligned} s_i[n] &= x[n + iH] \cdot w[n] \ , \\ n &= 0, \ldots, N-1 \ , \end{aligned} \tag{2.10}$$

donde H es el salto entre tramas (*hop size*).

Etapa 3: Transformada Rápida de Fourier (FFT)

Se aplica la FFT a cada trama ventaneada para obtener el espectro de magnitud:

$$\begin{aligned} S_i[k] &= \left| \sum_{n=0}^{N-1} s_i[n] e^{-j\frac{2\pi kn}{N}} \right|^2 \ , \\ k &= 0, \ldots, \frac{N}{2} \ . \end{aligned} \tag{2.11}$$

Solo se conservan los primeros $N/2+1$ bins (frecuencias positivas) debido a la simetría de la FFT para señales reales.

Etapa 4: Banco de Filtros Mel

El **banco de filtros Mel** es el componente que implementa la escala perceptual. Consiste en un conjunto de M filtros triangulares solapados, distribuidos en escala Mel, que cubren el rango de frecuencias de interés.

Construcción del banco de filtros:

1. **Definir rango**: Frecuencias mínima f_{min} y máxima f_{max} (ej: 0 Hz a $f_s/2$).

2. **Convertir a escala Mel**:

$$\begin{aligned} m_{min} &= 2595 \log_{10}\left(1 + \frac{f_{min}}{700}\right) \ , \\ m_{max} &= 2595 \log_{10}\left(1 + \frac{f_{max}}{700}\right) \ . \end{aligned}$$

3. **Dividir uniformemente en Mel**: Crear $M+2$ puntos equiespaciados

$$\begin{aligned} m[b] &= m_{min} + \frac{b}{M+1}(m_{max} - m_{min}) \ , \\ b &= 0, \ldots, M+1 \ . \end{aligned}$$

4. **Convertir de vuelta a Hz**:

$$f[b] = 700\left(10^{m[b]/2595} - 1\right) \ .$$

5. **Mapear a bins de FFT**:

$$\text{bin}[b] = \left\lfloor \frac{(N+1) \cdot f[b]}{f_s} \right\rfloor .$$

6. **Construir filtros triangulares**: Para cada filtro m ($m = 1, \ldots, M$):

$$H_m[k] = \begin{cases} 0 & k < \text{bin}[m-1] \\ \frac{k - \text{bin}[m-1]}{\text{bin}[m] - \text{bin}[m-1]} & \text{bin}[m-1] \leq k < \text{bin}[m] \\ \frac{\text{bin}[m+1] - k}{\text{bin}[m+1] - \text{bin}[m]} & \text{bin}[m] \leq k < \text{bin}[m+1] \\ 0 & k \geq \text{bin}[m+1] \end{cases}$$

Los filtros se aplican al espectro de potencia:

$$E_m = \sum_{k=0}^{N/2} S[k] \cdot H_m[k] \ , \quad m = 1, \ldots, M \ .$$

Parámetros típicos:

- $M = 20 - 40$ filtros (26-40 es común).
- $f_{min} = 0$ Hz o 20 Hz.
- $f_{max} = f_s/2$ o valor menor si se conoce BW del contenido.

Etapa 5: Logaritmo

Se aplica logaritmo para:

1. Comprimir el rango dinámico.
2. Aproximar la percepción logarítmica de sonoridad humana.
3. Convertir productos espectrales en sumas (separación de fuente/filtro).

$$\log E_m = \log \left(\sum_k S[k] \cdot H_m[k] \right) \ , \quad m = 1, \ldots, M \ .$$

Típicamente se usa $\log_{10}$ o ln. Para evitar $\log(0)$:

$$\log E_m = \log(\text{máx}(E_m, \epsilon)) \ .$$

donde ϵ es un valor pequeño (ej: 10^{-10}).

Etapa 6: Transformada Discreta del Coseno (DCT)

La **DCT** decorrela las energías de los bancos de filtros Mel, que tienden a estar correlacionadas debido al traslape entre filtros.

La DCT de tipo II se define como:

$$c[n] = \sum_{m=1}^{M} \log E_m \cdot \cos \left[\frac{\pi n (m - 0{,}5)}{M} \right] \ , \quad n = 0, \ldots, M-1 \ .$$

Los coeficientes resultantes $c[n]$ son los **MFCC**. Típicamente se conservan solo los primeros C coeficientes (usualmente $C = 12$ o 13):

$$\text{MFCC} = [c_0, c_1, c_2, \ldots, c_{C-1}] \ .$$

Interpretación física:

- c_0: Relacionado con energía promedio (a veces se descarta o reemplaza por log-energía).
- c_1, c_2: Capturan envolvente espectral

gruesa.

- $c_3, \ldots, c_{12}$: Detalles espectrales más finos.
- $c_{>12}$: Usualmente desechados (información menos relevante, más ruido).

Etapa 7: Características Dinámicas (Δ y Δ^2)

Los MFCC estáticos capturan información espectral instantánea pero ignoran la dinámica temporal. Las **derivadas temporales** capturan estos cambios:

Delta (velocidad): primera derivada, captura tasa de cambio

$$\Delta c[n,t] = \frac{\sum_{\theta=1}^{\Theta} \theta(c[n,t+\theta] - c[n,t-\theta])}{2\sum_{\theta=1}^{\Theta} \theta^2} .$$

Delta-Delta (aceleración): segunda derivada

$$\Delta^2 c[n,t] = \frac{\sum_{\theta=1}^{\Theta} \theta(\Delta c[n,t+\theta] - \Delta c[n,t-\theta])}{2\sum_{\theta=1}^{\Theta} \theta^2} ,$$

donde Θ es la ventana de cálculo (típicamente $\Theta = 2$).

El **vector de características completo** para cada trama es:

$$\mathbf{f}_t = [\mathrm{MFCC}_t, \Delta\mathrm{MFCC}_t, \Delta^2\mathrm{MFCC}_t] .$$

Si se usan 13 MFCC, el vector tiene dimensión $13 \times 3 = 39$ características.

2.4.4. Configuraciones de MFCC en Estudios de Colmenas

Ruvinga et al. [4, 9] evaluaron 4 configuraciones:

Resultados clave:

- Dataset 1 (14 características) dio **mejores resultados**.
- c_0 (relacionado con energía DC) y log-energía son informativos.
- Dataset 4 (12 características, sin c_0 ni log-energía) dio peores resultados.

Esto sugiere que tanto la información de energía global como la estructura armónica completa son relevantes para discriminar estados de colmena.

2.4.5. Implementación Práctica

Librerías ampliamente usadas para MFCC:

- **Python**: `librosa.feature.mfcc()`, `python_speech_features`.
- **MATLAB**: `mfcc()` en Audio Toolbox.
- **C++**: Bibliotecas como `Essentia`, `Aquila`.

Parámetros típicos para audio de colmenas basados en tu trabajo:

```python
import librosa

mfcc = librosa.feature.mfcc(
    y=audio_signal,
    sr=22050,               # Frecuencia de muestreo
    n_mfcc=13,              # Numero de coeficientes
    n_fft=2048,             # Tamano de FFT
    hop_length=512,         # Salto entre tramas
    n_mels=40,              # Numero de filtros Mel
    fmin=0,                 # Frecuencia minima
    fmax=8000               # Frecuencia maxima
```

```
)
```

2.5. Aprendizaje Profundo para Clasificación de Audio

2.5.1. Conceptos Fundamentales de Redes Neuronales

Las **redes neuronales artificiales (ANN)** son modelos computacionales inspirados en el sistema nervioso biológico, compuestos por unidades interconectadas (neuronas artificiales) organizadas en capas.

Perceptrón y Arquitectura Básica

Una neurona artificial implementa:

$$y = \phi\left(\sum_{i=1}^{n} w_i x_i + b\right) = \phi(\mathbf{w}^T\mathbf{x} + b) \ ,$$

donde:

- $\mathbf{x} = [x_1, \ldots, x_n]^T$: vector de entrada.
- $\mathbf{w} = [w_1, \ldots, w_n]^T$: pesos sinápticos.
- b: sesgo (*bias*).
- $\phi(\cdot)$: función de activación no lineal.

Funciones de Activación

Sigmoide (Logística):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1) \ ,$$

usada en capas de salida para clasificación binaria.

Tangente Hiperbólica:

$$\tanh(z) = \frac{e^{2z} - 1}{e^{2z} + 1} \in (-1, 1) \ ,$$

preferida sobre sigmoide en capas ocultas por estar centrada en cero.

ReLU (Rectified Linear Unit):

$$\mathrm{ReLU}(z) = \text{máx}(0, z) \ ,$$

actualmente la más popular por:

- Eficiencia computacional.
- Mitigación del problema de gradientes desvanecientes.
- Inducción de escasez (sparsity).

Softmax (para clasificación multiclase):

$$\mathrm{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \ ,$$

donde K es el número de clases.

Entrenamiento: Retropropagación y Optimización

El entrenamiento consiste en ajustar $\mathbf{w}$ y b para minimizar una función de pérdida $\mathcal{L}$.

Función de pérdida para clasificación binaria (Entropía Cruzada):

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)] \ .$$

El algoritmo de **retropropagación** calcula gradientes:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_{ij}} \ ,$$

y los parámetros se actualizan mediante **descenso de gradiente**:

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}} \ ,$$

donde η es la tasa de aprendizaje.

2.5.2. Perceptrón Multicapa (MLP)

Un MLP es una red feedforward con al menos una capa oculta. Para clasificación binaria de estados de colmena:

- **Capa de entrada**: n neuronas (dimensión de MFCC, ej: 14).
- **Capas ocultas**: L capas con $h_1, \ldots, h_L$ neuronas cada una.
- **Capa de salida**: 1 neurona con activación sigmoide.

Ruvinga et al. [9] reportaron:

- MLP con 14 MFCC de entrada.
- Función de activación: tangente hiperbólica en capas ocultas.
- Precisión: 90.1 % (Dataset 1).

2.5.3. Regresión Logística

Puede verse como un MLP sin capas ocultas (modelo lineal + sigmoide):

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) \ .$$

Ruvinga et al.: Precisión 87.4 % con 14 MFCC.

Limitación: Asume separabilidad lineal en el espacio de características.

2.5.4. Redes Neuronales Recurrentes (RNN) y LSTM

Limitaciones de MLP para Secuencias

Los MLP procesan cada trama de forma independiente, ignorando:

- Dependencias temporales entre tramas.
- Contexto histórico de la señal.
- Patrones secuenciales.

Para audio, esto es subóptimo porque el significado depende de la evolución temporal.

Redes Neuronales Recurrentes (RNN)

Las RNN mantienen un **estado oculto** que se actualiza en cada paso temporal:

$$\begin{aligned} a_t &= \tanh(W_{hh}a_{t-1} + W_{xh}x_t + b_h) \ , \\ y_t &= W_{hy}a_t + b_y \ . \end{aligned}$$

Ventajas:

- Memoria de eventos pasados.
- Parámetros compartidos en el tiempo (generalización).

Problema crítico: **Desvanecimiento/explosión de gradientes**.
Durante retropropagación a través del tiempo (BPTT), los gradientes se calculan como productos de derivadas:

$$\frac{\partial \mathcal{L}}{\partial a_1} = \frac{\partial \mathcal{L}}{\partial a_T} \prod_{t=2}^{T} \frac{\partial a_t}{\partial a_{t-1}} \ .$$

Si $\|\frac{\partial a_t}{\partial a_{t-1}}\| < 1$: gradientes desvanecen exponencialmente.

Si $\|\frac{\partial a_t}{\partial a_{t-1}}\| > 1$: gradientes explotan.

Consecuencia: Incapacidad de aprender dependencias a largo plazo.

Long Short-Term Memory (LSTM)

Las **LSTM** [18] resuelven el problema de gradientes mediante una arquitectura especializada que fuerza flujo constante de error.

Arquitectura de la Célula LSTM:

- **Estado de celda** c_t: "memoria"de largo plazo.
- **Estado oculto** a_t: salida de corto plazo.
- **Tres compuertas** que controlan flujo de información:

- Compuerta de olvido f_t.
- Compuerta de entrada i_t.
- Compuerta de salida o_t.

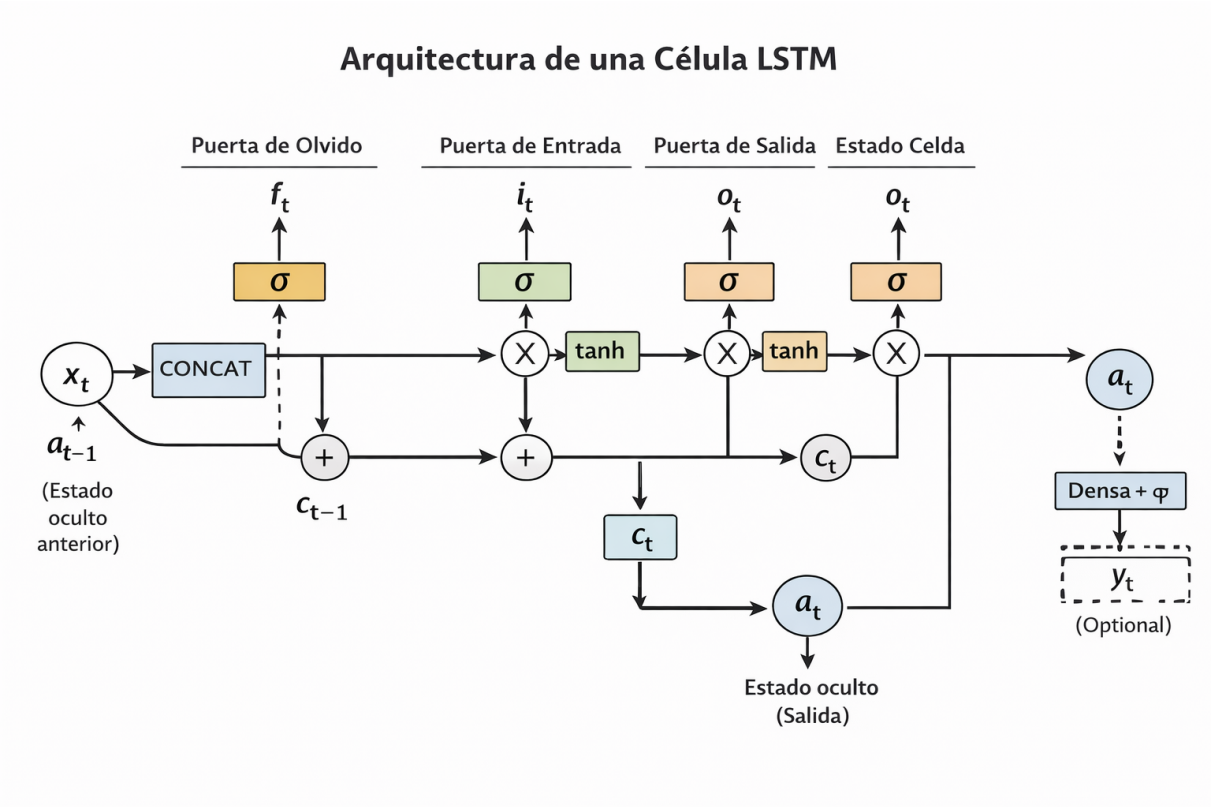


Figura 2.3: Arquitectura de una célula LSTM mostrando las tres compuertas y los estados de celda.

Ecuaciones de la LSTM:

$$
\begin{aligned}
f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) && \text{(compuerta de olvido).}\\
i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) && \text{(compuerta de entrada).}\\
\tilde{c}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) && \text{(candidato a estado de celda).}\\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{(actualización del estado).}\\
o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) && \text{(compuerta de salida).}\\
a_t &= o_t \odot \tanh(c_t) && \text{(estado oculto de salida).}
\end{aligned}
$$

donde:

- σ: función sigmoide (salidas en $[0, 1]$).
- $\odot$: producto de Hadamard (elemento a elemento).

Interpretación de las Compuertas:

1. **Compuerta de olvido** ($f_t \in [0, 1]$):

- Decide qué información del estado previo c_{t-1} descartar.
- $f_t \approx 0$: olvida completamente.
- $f_t \approx 1$: retiene completamente.

2. **Compuerta de entrada** ($i_t \in [0, 1]$):
   - Decide qué nueva información almacenar en c_t.
   - Filtra el candidato $\tilde{c}_t$.
   - $i_t \approx 0$: ignora nueva información.
   - $i_t \approx 1$: incorpora completamente.
3. **Compuerta de salida** ($o_t \in [0, 1]$):
   - Decide qué partes del estado de celda c_t exponer como salida a_t.

Flujo de Gradientes: la clave del éxito de LSTM está en la actualización del estado de celda:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \ .$$

El gradiente fluye como:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \ .$$

Si $f_t \approx 1$, el gradiente fluye sin atenuación (evita desvanecimiento).

LSTM para Clasificación de Estados de Colmena

Ruvinga et al. [4, 9] implementaron:

- **Entrada**: secuencia de vectores MFCC $\mathbf{X} = [x_1, x_2, \ldots, x_T]$ donde $x_t \in \mathbb{R}^{14}$.
- **Arquitectura**:
  - 1 capa LSTM con 100 unidades.
  - Capa densa (fully connected) con 1 neurona + sigmoide.
- **Función de pérdida**: entropía cruzada binaria.
- **Optimizador**: ADAM (Adaptive Moment Estimation).
  - Combina momentum y RMSProp.
  - Tasas de aprendizaje adaptativas por parámetro.
  - Eficiente computacionalmente.
- **Resultados**:
  - Precisión: 92 % (validación).
  - F1-Score: 92 %.
  - Superó a MLP (90 %) y Regresión Logística (87 %).

Ventajas de LSTM sobre MLP:

- Captura patrones temporales en la evolución del sonido.
- Detecta transiciones entre estados acústicos.
- Más robusto a variaciones temporales naturales.

2.5.5. Redes Neuronales Convolucionales (CNN)

Motivación para Audio

Mientras LSTM opera sobre secuencias de vectores (MFCC), las CNN procesan representaciones bidimensionales como espectrogramas, tratándolos como "imágenes" tiempo-frecuencia.

Ventajas de CNN:

- Detección automática de características jerárquicas.
- Invarianza a traslaciones temporales y frecuenciales.
- Uso eficiente de parámetros (compartición de pesos).
- Capacidad de detectar patrones locales y globales.

Arquitectura de CNN

Una CNN típica consta de:

1. **Capas Convolucionales**: Detectan características locales.
2. **Capas de Activación**: Introducen no linealidad (ReLU).
3. **Capas de Pooling**: Reducen dimensionalidad espacial.
4. **Capas Densas (Fully Connected)**: Clasificación final.

Operación de Convolución 2D: para una entrada I (espectrograma) y un kernel K de tamaño $m \times n$,

$$(I * K)(i,j) = \sum_{p=0}^{m-1} \sum_{q=0}^{n-1} I(i+p, j+q) \cdot K(p,q) \ .$$

Cada filtro produce un **mapa de características** (feature map).

Capa Convolucional:

Con C filtros, hiperparámetros:

- Número de filtros C (ej: 32, 64, 128).
- Tamaño de kernel $f \times f$ (ej: 3×3, 5×5).
- Stride s (desplazamiento del kernel).
- Padding p (relleno de bordes).

Dimensiones de salida:

$$w_{out} = \left\lfloor \frac{w_{in} - f + 2p}{s} \right\rfloor + 1 \ ,$$
$$a_{out} = \left\lfloor \frac{a_{in} - f + 2p}{s} \right\rfloor + 1 \ .$$

Capa de Pooling: reduce dimensión espacial, reteniendo información relevante.

Max Pooling: Toma el valor máximo en cada región

$$y_{ij} = \max_{(p,q) \in R_{ij}} x_{pq} \ .$$

Average Pooling: Toma el promedio

$$y_{ij} = \frac{1}{|R_{ij}|} \sum_{(p,q) \in R_{ij}} x_{pq} \ .$$

Tamaño típico: 2×2 con stride 2 (reduce dimensión a la mitad).

CNN para Clasificación de Colmenas

Ruvinga et al. [9] implementaron:

- **Entrada**: Espectrogramas STFT de 64×64 píxeles.
- **Arquitectura**:
  - Capa Conv2D: 64 filtros, kernel 3×3, ReLU.
  - MaxPooling: 2×2.
  - Capa Conv2D: 128 filtros, kernel 3×3, ReLU.
  - MaxPooling: 2×2.
  - Flatten (aplanar).
  - Capa densa: N neuronas, ReLU.
  - Capa de salida: 1 neurona, sigmoide.
- **Preprocesamiento**:
  - Segmentos de audio de 5 segundos.
  - Espectrogramas calculados con STFT.
  - Normalización de píxeles.
- **Resultados**:
  - **Precisión (training)**: 99.12 %.
  - **Precisión (validación)**: 99.31 %.
  - **Precisión (test, dataset independiente)**: 99 %.
  - **Recall**: 99.31 %.

- **F1-Score**: 99.31 %.

Superioridad de CNN:

- Captura patrones espacio-temporales en espectrogramas.
- Detecta texturas acústicas características.
- Aprende representaciones jerárquicas automáticamente.
- Menor sensibilidad a variaciones locales (pooling).

2.5.6. Comparación de Arquitecturas

Lecciones clave:

1. Modelado temporal (LSTM) ¿Estático (MLP, LR).
2. Representación rica (espectrograma) ¿Compacta (MFCC).
3. Aprendizaje jerárquico (CNN) muy efectivo para patrones complejos.
4. Trade-off: Precisión vs. complejidad computacional.

2.6. Síntesis y Justificación Metodológica

Este capítulo ha establecido los fundamentos teóricos para la detección automática del estado de *queenlessness* en colmenas mediante análisis acústico y aprendizaje profundo. Los principales hallazgos de la revisión son:

2.6.1. Contexto y Motivación

1. Las abejas melíferas son polinizadores críticos, contribuyendo USD $235-577 mil millones anuales a la agricultura global.
2. El declive poblacional documentado en múltiples regiones amenaza la seguridad alimentaria.
3. La reina es esencial para la supervivencia de la colonia; su pérdida lleva a extinción si no se detecta tempranamente.
4. El monitoreo tradicional es invasivo, laborioso y riesgoso.

2.6.2. Estado del Arte en Monitoreo Acústico

1. Las abejas producen señales acústicas informativas sobre el estado de la colonia.
2. Estudios previos han identificado características espectrales distintivas de *queenlessness* (especialmente 165-285 Hz).
3. Técnicas de machine learning han mostrado precisiones de 87-92 % usando MFCC.
4. Deep learning (CNN) ha alcanzado hasta 99 % de precisión usando espectrogramas.
5. Persisten brechas en generalización, robustez al ruido, e implementación en sistemas embebidos.

2.6.3. Fundamentos Técnicos

1. **MFCC** proporcionan representación compacta y robusta del contenido espectral, capturando información perceptualmente relevante.
2. **LSTM** capturan dependencias temporales a largo plazo, superando a modelos estáticos.
3. **CNN** procesan espectrogramas como imágenes tiempo-frecuencia, aprendiendo características jerárquicas automáticamente.

4. La elección de características (MFCC vs. espectrograma) y arquitectura (LSTM vs. CNN) implica trade-offs entre precisión, interpretabilidad y costo computacional.

2.6.4. Justificación de la Metodología Propuesta

Basándose en esta revisión, el presente estudio adoptará:

- **MFCC como características primarias**: Por su compacidad, robustez y éxito demostrado en la literatura.
- **Evaluación de múltiples arquitecturas**: LSTM y potencialmente CNN para comparación exhaustiva.
- **Énfasis en preprocesamiento**: Segmentación, normalización, y posible denoising para mejorar robustez.
- **Validación rigurosa**: División train/validation/test con datos de múltiples colmenas y condiciones cuando sea posible.
- **Consideración de implementación**: Evaluación de viabilidad en sistemas embebidos de bajo costo.

El siguiente capítulo detallará la metodología específica implementada, incluyendo el dataset utilizado, preprocesamiento de señales, extracción de características, diseño de modelos y métricas de evaluación.

3. METODOLOGÍA

3.1. Procedimiento

Como verificación inicial, se ejecutó un programa de prueba (programa A.1) para la extracción de características acústicas utilizando coeficientes cepstrales en escala Mel (MFCC) y validar la conversión del audio a representaciones numéricas (dimensionalidad esperada, ausencia de valores indefinidos y consistencia temporal), antes de procesar el conjunto completo.

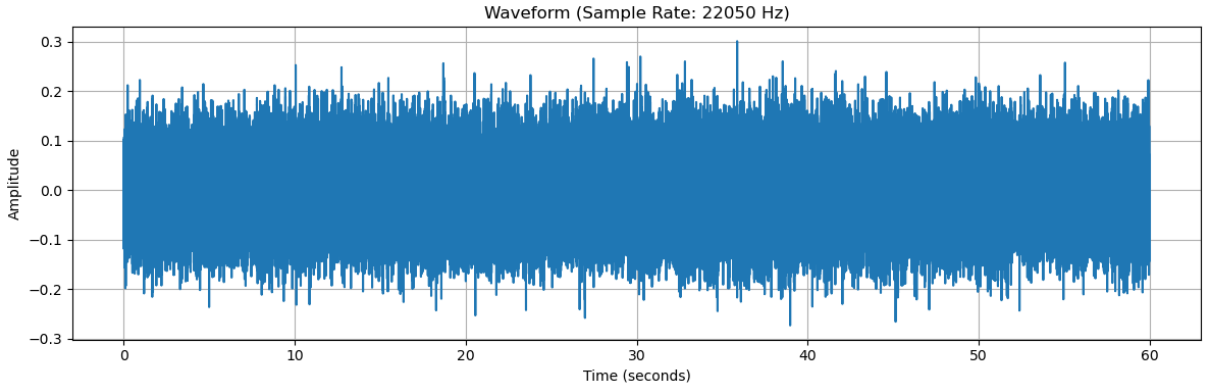


Figura 3.1: Forma de onda del archivo de audio `2022-06-05--19-39-14_2__segment2.wav`.

En este estudio se trabajó con audio monofónico (1 canal) en formato .WAV (figura 3.1) cuyos metadatos indican una frecuencia de muestreo de 22050 Hz y muestras `float32` (en punto flotante de 32 bits). Para estandarizar el procesamiento y reducir el costo computacional, todas las señales se remuestrearon a 16000 Hz durante la carga en Python.

El flujo de procesamiento se validó por etapas en cuadernos `*.ipynb` y posteriormente se automatizó para el conjunto completo: carga, normalización, segmentación, extracción de características, generación de arreglos `*.npy` para el entrenamiento, desarrollo y aplicación del modelo.

Tras la carga y remuestreo a 16000 Hz, se aplicó un recorte de silencios con un umbral de `top_db = 30`. Luego, cada señal recortada se normalizó, escalando la amplitud de modo que el valor absoluto máximo fuese igual a 1,0:

$$x_{\text{norm}}[n] = \frac{x_{\text{trim}}[n]}{A_{\max}}$$

donde $A_{\max} = \text{máx}_m \, |x_{\text{trim}}[m]|$ es el valor máximo absoluto de la señal recortada y $x_{\text{trim}}[n]$ es la señal después del recorte de silencios. Esta normalización garantiza que la señal resultante tenga valores en el rango $[-1, 1]$. Al aplicarse a nivel de archivo, reduce el efecto de diferencias globales de ganancia entre grabaciones sobre la extracción de características espectrales, permitiendo que el modelo se enfoque en patrones relevantes para la clasificación del estado de la reina.

Finalmente, cada señal normalizada se segmentó en ventanas de 2 segundos con un desplazamiento (hop) de 1 segundo, como se muestra en el ejemplo de la figura 3.2 (a) y (b), lo que equivale a un solapamiento del 50 % entre seg-

mentos consecutivos. Esta segmentación incrementa el número de ejemplos disponibles para el entrenamiento y, al mismo tiempo, produce entradas de duración fija para el modelo. Una vez obtenidos los segmentos de 2 segundos, se procedió a la extracción de características espectrales mediante coeficientes cepstrales en escala Mel (MFCC).

Para cada segmento, se calcularon MFCC utilizando una ventana de análisis de 25 ms (`win_length` $= 0{,}025 \cdot 16000 = 400$ muestras), con un tamaño de FFT de $N_{\text{FFT}} = 512$ puntos (potencia de 2 inmediatamente superior), y un desplazamiento temporal de 10 ms (`hop_length` $= 0{,}010 \cdot 16000 = 160$ muestras), como en la figura 3.2 (c). En el dominio espectral, para la construcción del mel-spectrogram, la energía se analizó en el rango de frecuencias de 20 Hz a 8000 Hz (frecuencia de Nyquist para la tasa de muestreo de 16000 Hz), utilizando 40 filtros en escala Mel. Esta configuración permite capturar la distribución espectral con mayor resolución en bajas frecuencias, donde se esperaba encontrar patrones acústicos relevantes para la clasificación del estado de la reina.

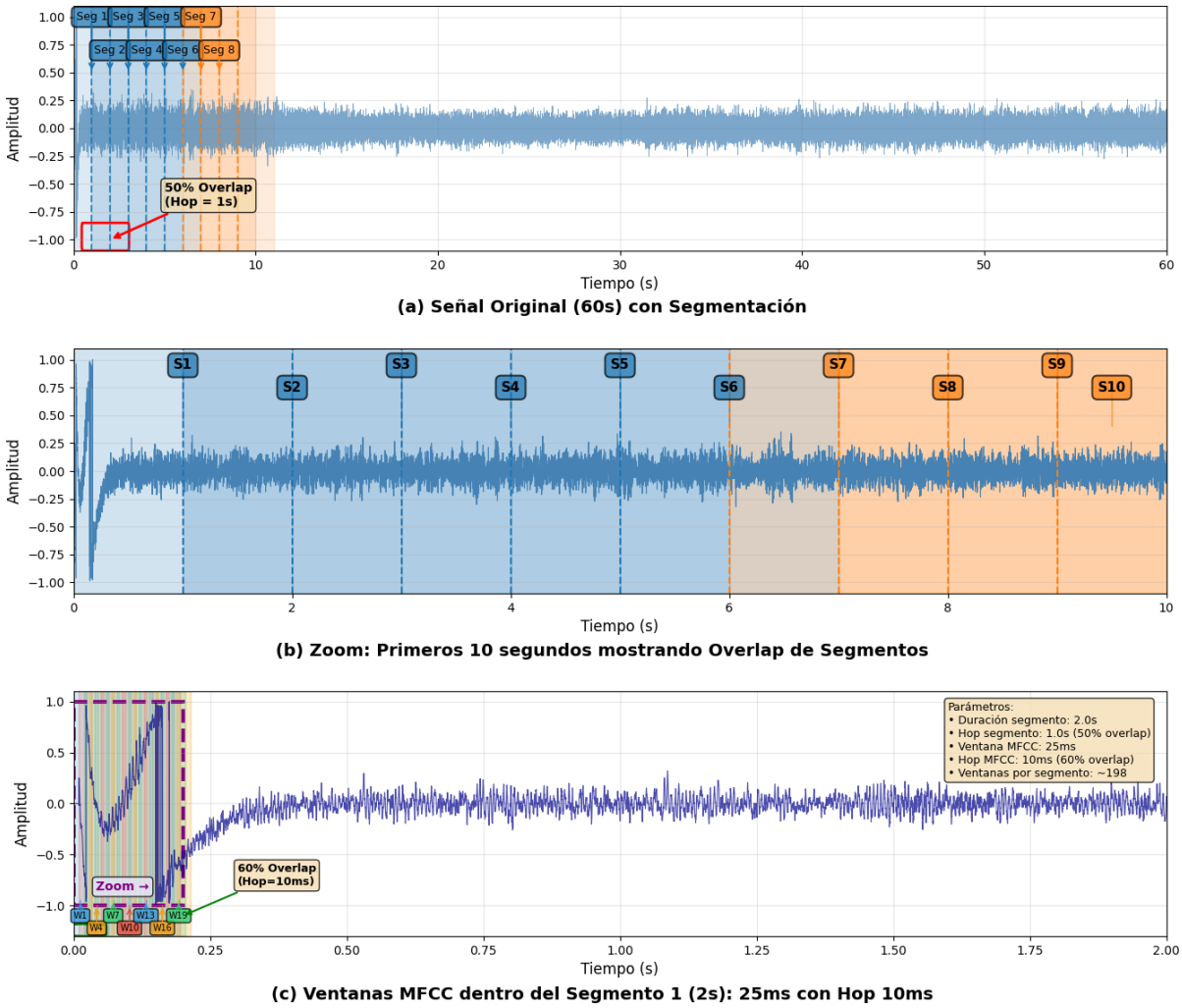


Figura 3.2: Ejemplo de segmentación de una señal de audio y ventanas MFCC.

En la figura 3.3 se muestra un acercamiento de las primeras 10 ventanas MFCC que fueron expuestas en la figura 3.2 (c), donde se puede observar la representación de cada ventana de 25 ms con un desplazamiento de 10 ms y un solapamiento del 60 % entre ventanas consecutivas. Esta configuración permite capturar la evolución temporal de las características espectrales, lo que es crucial para la tarea de clasificación del estado de la reina, ya que se esperaba que ciertos patrones acústicos se manifestaran de manera dinámica a lo largo del tiempo dentro de la colmena.

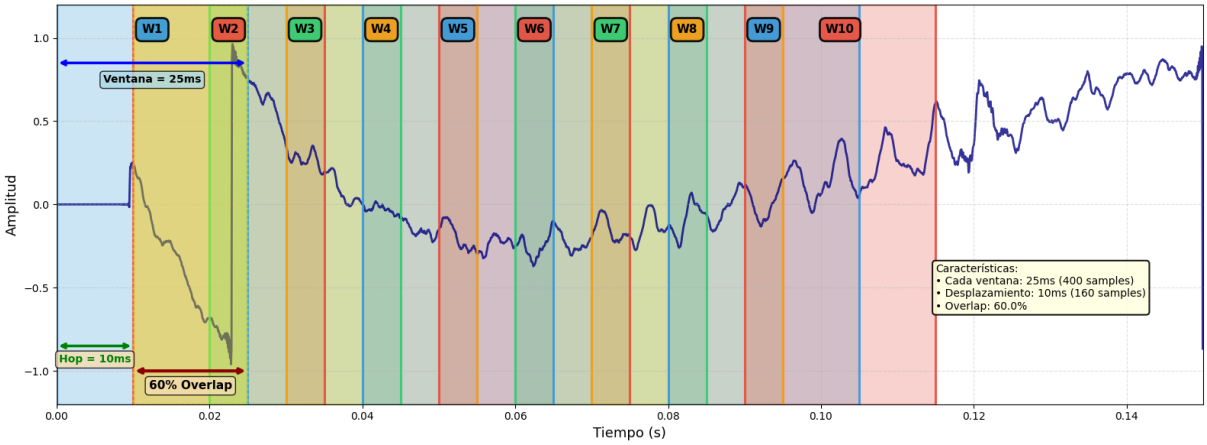


Figura 3.3: ZOOM: Primeras 10 ventanas MFCC (25 ms con Hop 10 ms) - 60 % Overlap.

Posteriormente, mediante la Transformada Discreta del Coseno (DCT), se obtuvieron coeficientes cepstrales que comprimen esta información espectral. Para los audios involucrados en el estudio, se calcularon inicialmente 33 coeficientes MFCC y posteriormente se eliminó el coeficiente C_0, que representa la energía promedio del espectro y puede ser menos informativo para la clasificación especialmente en presencia de variaciones de intensidad entre grabaciones; resultando en 32 coeficientes finales (ver figura 3.4 (a)). Adicionalmente, se incluyeron las derivadas temporales de primer y segundo orden (Δ y Δ^2) para capturar dinámica temporal (figura 3.4 (b) y (c)), obteniendo una representación final de tamaño $(3 \times 32 \times T)$ por segmento, donde T es el número de cuadros temporales en el segmento (esta fue la representación que se organizó en arreglos `*.npy` para su uso posterior en el entrenamiento del modelo de aprendizaje profundo).

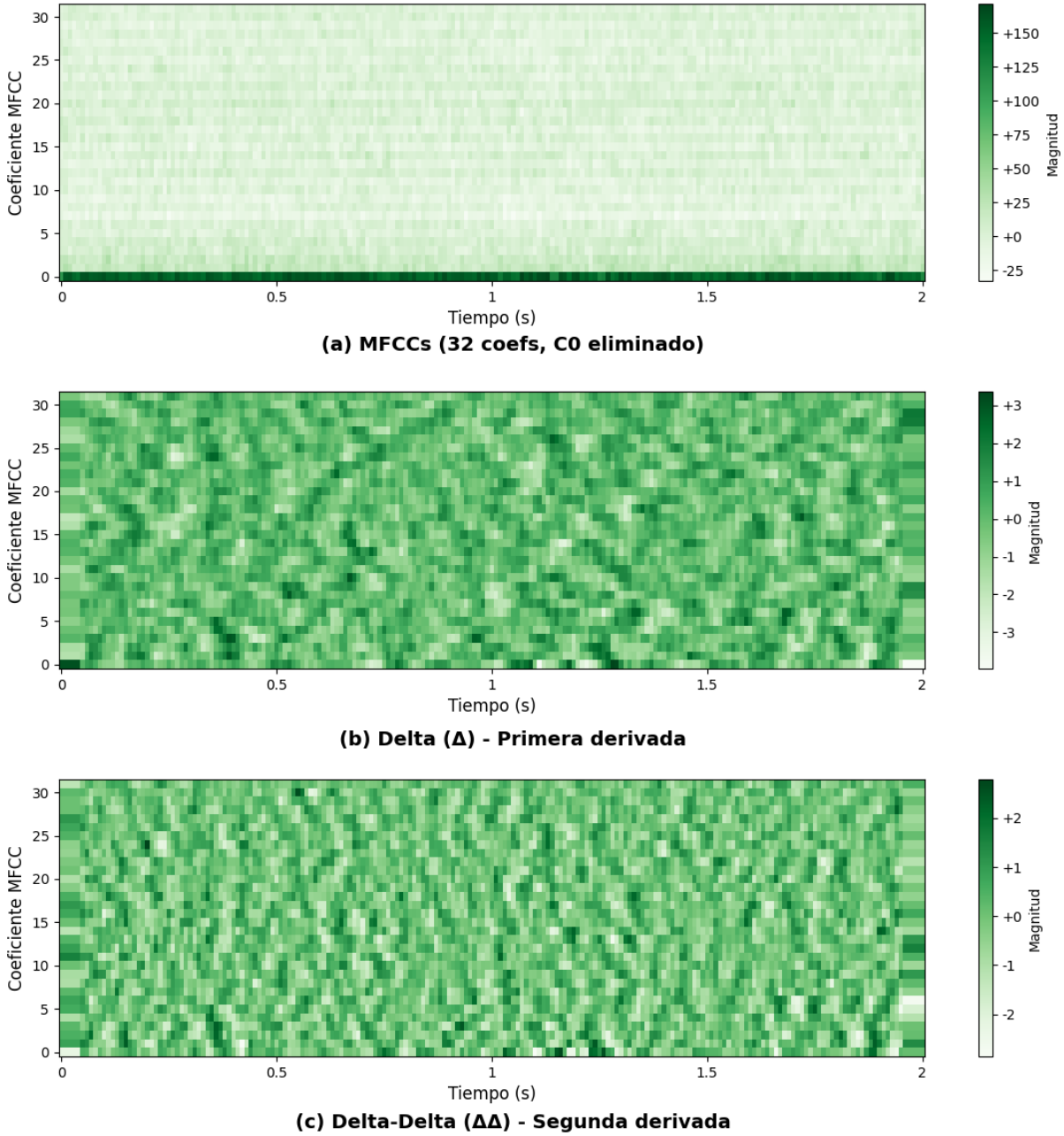


Figura 3.4: Ejemplo de estas tres capas de características para un segmento de 2 segundos.

La señal de audio para uno de los archivos, como el de la figura 3.1, se representa de forma discreta. Una frecuencia de muestreo de 22050 Hz indica que el audio se capturó mediante 22050 muestras por segundo. A partir del análisis en Python, se observó que este archivo contiene un vector unidimensional de $N = 1{,}323{,}000$ muestras, por lo que su duración aproximada es

$$N/f_s = 1{,}323{,}000/22{,}050 \approx 60 \text{ segundos.}$$

Cada muestra corresponde a un valor numérico (almacenado como `float32`) que representa la amplitud instantánea de la señal. Matemáticamente, esta señal discreta puede formalizarse

como:

$$x[n] = \{x_0, x_1, \ldots, x_{N-1}\} \ ,$$
$$n = 0, 1, \ldots, N-1 \ ;$$

donde $x[n]$ representa la amplitud en el instante discreto n y N es el número total de muestras. En consecuencia, un archivo de audio puede interpretarse como una secuencia de valores que describe cómo varía la presión del aire (sonido) a lo largo del tiempo.

No obstante, para el entrenamiento de un modelo de aprendizaje profundo, esta representación temporal no es la más conveniente para capturar patrones espectrales relevantes en la clasificación del estado de la reina. Por ello, la señal se transforma a una representación tiempo–frecuencia mediante el cálculo de espectros por ventanas (STFT), se proyecta su energía según bandas en escala Mel y se aplica una compresión logarítmica; posteriormente, se obtiene un conjunto compacto de descriptores mediante la Transformada Discreta del Coseno (DCT), resultando en los coeficientes cepstrales en escala Mel (MFCC). Esta representación puede interpretarse como una matriz (análoga a una imagen), lo cual facilita el uso de redes neuronales convolucionales (CNN) para la tarea de clasificación.

3.2. Arquitectura del Modelo

Se implementó una red neuronal convolucional (CNN) para la clasificación multiclase del estado de la reina en colmenas de abejas melíferas. El modelo recibe como entrada tensores de dimensión $(3, 32, 201)$, correspondientes a:

- 3 canales: MFCC estáticos, derivadas de primer orden (Δ) y derivadas de segundo orden (Δ^2).
- 32 coeficientes cepstrales (excluyendo C_0).
- 201 tramas temporales (equivalentes a segmentos de audio de 2 segundos).

La arquitectura implementada consta de cuatro bloques convolucionales con normalización por lotes y *max-pooling*, seguidos de capas densas para la clasificación final. La tabla 3.1 resume la estructura del modelo.

Cuadro 3.1: **Arquitectura de la red neuronal convolucional implementada**

Capa	Tipo	Forma de Salida	Parámetros
Input	–	$(3, 32, 201)$	–
Conv2D_1	Conv2D + Sigmoid	$(16, 32, 201)$	448
BatchNorm_1	Normalización	$(16, 32, 201)$	64
MaxPool_1	Max Pooling 2×2	$(16, 16, 100)$	0
Conv2D_2	Conv2D + Sigmoid	$(32, 16, 100)$	4,640
BatchNorm_2	Normalización	$(32, 16, 100)$	128
MaxPool_2	Max Pooling 2×2	$(32, 8, 50)$	0
Conv2D_3	Conv2D + Sigmoid	$(64, 8, 50)$	18,496
BatchNorm_3	Normalización	$(64, 8, 50)$	256
MaxPool_3	Max Pooling 2×2	$(64, 4, 25)$	0
Conv2D_4	Conv2D + Sigmoid	$(128, 4, 25)$	73,856
BatchNorm_4	Normalización	$(128, 4, 25)$	512
GlobalAvgPool	Promedio Global	(128)	0
Dense_1	Fully Connected + ReLU	(64)	8,256
Dropout	Regularización (p=0.3)	(64)	0
Dense_2	Softmax	(4)	260
		Total de parámetros:	**106,916**

El modelo fue compilado con las siguientes características:

- **Optimizador**: Adam con tasa de aprendizaje inicial de 1×10^{-3}.
- **Función de pérdida**: entropía cruzada categórica dispersa (*sparse categorical crossentropy*).
- **Métrica**: precisión (*accuracy*).
- **Regularización**: pesos de clase para compensar el desbalance en el conjunto de entrenamiento (clase 0: 1.78, clase 1: 2.01, clase 2: 1.23, clase 3: 0.47).

3.3. Distribución del Conjunto de Datos

El conjunto de datos procesado resultó en un total de 75,112 segmentos de audio de 2 segundos, distribuidos de la siguiente manera:

Cuadro 3.2: **Distribución de segmentos por conjunto y clase**

Conjunto	Clase 0	Clase 1	Clase 2	Clase 3	Total
Entrenamiento	7,368	6,535	10,668	27,980	52,551
Validación	1,582	1,400	2,289	5,982	11,253
Prueba	1,591	1,413	2,299	6,005	11,308
Total	**10,541**	**9,348**	**15,256**	**39,967**	**75,112**

donde:

- **Clase 0**: Reina original/funcional (*queen-right*).
- **Clase 1**: Reina no presente (*queenless*).
- **Clase 2**: Reina presente pero rechazada.
- **Clase 3**: Reina presente y recién aceptada.

3.4. Instrumentos

Para el proceso de recolección de datos, se empleó un dispositivo IoT diseñado a medida, que incluía un módulo Wi-Fi `ESP32`, un micrófono digital `INMP441`, y un sensor de temperatura y humedad `BME280`. Estos sensores se instalaron dentro de la cubierta telescópica de la colmena —por encima de los cuadros pero por debajo de la cubierta exterior—, y se protegieron con una malla metálica para evitar la interferencia de las abejas con los componentes electrónicos, especialmente el micrófono.

3.5. Base de Datos

El conjunto de datos utilizado en esta investigación fue grabado y publicado por Anna Yang[19]. Contiene datos de audio recolectados de colmenas de abejas melíferas europeas ubicadas en California. Las grabaciones fueron segmentadas en clips de 60 segundos, resultando en un total de 7,100 muestras recolectadas durante un período de cinco semanas, desde el 8 de junio de 2022 (14:52:28) hasta el 15 de julio de 2022 (15:28:21). Cada día contiene aproximadamente entre 23 y 24 muestras.

El conjunto de datos incluye múltiples variables de metadatos que describen el entorno de la colmena y la condición de la colonia. De particular relevancia para este trabajo se encuentran las siguientes etiquetas:

- ☞ `queen acceptance`: indica si la reina es aceptada por la colonia. Valores: 0 (sin reina), 1 (presente pero no aceptada), 2 (aceptada).
- ☞ `queen presence`: indica si la reina está físicamente presente en la colmena.
- ☞ `queen status`: una etiqueta combinada derivada de las dos anteriores, con los siguientes valores: 0 (original/con reina funcional), 1 (no presente), 2 (presente y rechazada), 3 (presente y recién aceptada). Esta es la etiqueta principal utilizada para la clasificación.
- ☞ `time`: hora del día, escalada entre 0 y 1 para representar un ciclo de 24 horas.

El proceso de etiquetado se basó en inspecciones manuales diarias de las colmenas, durante las cuales se observó el comportamiento de las abejas hacia la reina (por ejemplo, formación de enjambre o conducta tranquila). En aquellos casos en los que el estado de la reina cambió entre inspecciones, generando ambigüedad sobre el momento exacto de aceptación, las muestras correspondientes fueron descartadas para mantener la integridad del etiquetado.

4. RESULTADOS

4.1. Resultados del Entrenamiento

El modelo fue entrenado durante 40 épocas con mecanismos de parada temprana (*early stopping*) y reducción adaptativa de la tasa de aprendizaje (*learning rate scheduler*).

Los mejores resultados en el conjunto de validación se obtuvieron en la época 27, alcanzando:

- **Precisión de validación**: 87.71 %
- **Pérdida de validación**: 0.6462
- **Precisión de entrenamiento**: 99.21 %
- **Pérdida de entrenamiento**: 0.0176

El modelo mostró una tendencia de sobreajuste moderado, evidenciada por la diferencia entre la precisión de entrenamiento (99.21 %) y validación (87.71 %). Sin embargo, la aplicación de regularización mediante *dropout* y pesos de clase permitió mantener un desempeño aceptable.

4.2. Evaluación en el Conjunto de Prueba

El modelo final fue evaluado en el conjunto de prueba (11,308 segmentos), obteniendo una precisión global de **86.58 %**. La tabla 4.1 presenta la matriz de confusión, que permite analizar el desempeño del clasificador para cada clase.

Cuadro 4.1: **Matriz de confusión del modelo en el conjunto de prueba**

Clase Real	**Clase Predicha**				**Total**
	0	**1**	**2**	**3**	
0 (Queenright)	1,235	95	88	173	1,591
1 (No presente)	42	1,131	147	93	1,413
2 (Rechazada)	50	87	1,912	250	2,299
3 (Aceptada)	19	143	330	5,513	6,005
Total predicho	1,346	1,456	2,477	6,029	11,308

La matriz de confusión revela varios patrones importantes:

1. **Clase 0 (Queenright)**: el modelo clasificó correctamente 1,235 de 1,591 casos (77.62 %), con confusiones principalmente hacia la clase 3 (173 casos, 10.88 %).
2. **Clase 1 (No presente)**: alcanzó la mejor tasa de clasificación correcta con 1,131 de 1,413 casos (80.04 %), mostrando mayor dificultad para distinguirla de la clase 2 (147 confusiones).
3. **Clase 2 (Rechazada)**: obtuvo 1,912 clasificaciones correctas de 2,299 (83.17 %), con la mayoría de errores dirigidos hacia la clase 3 (250 casos).
4. **Clase 3 (Aceptada)**: mostró el mejor desempeño con 5,513 clasificaciones correctas de 6,005 (91.81 %), aunque confundió 330 casos con la clase 2.

La tabla 4.2 presenta las métricas de clasificación detalladas para cada clase.

Cuadro 4.2: **Métricas de clasificación por clase**

Clase	Precisión	Recall	F1-Score	Soporte
0 (Queenright)	0.9175	0.7762	0.8410	1,591
1 (No presente)	0.7768	0.8004	0.7884	1,413
2 (Rechazada)	0.7719	0.8317	0.8007	2,299
3 (Aceptada)	0.9144	0.9181	0.9162	6,005
Promedio macro	0.8452	0.8316	0.8366	–
Promedio ponderado	0.8687	0.8658	0.8662	11,308
Precisión global	0.8658			11,308

Las métricas revelan:

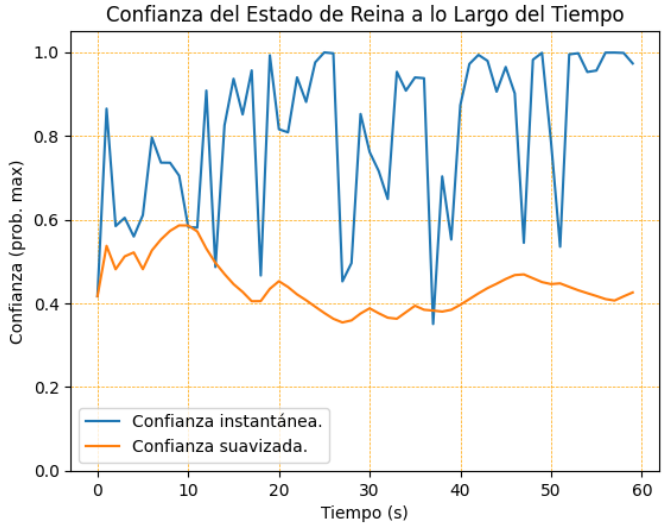
- **Precisión**: la clase 0 (queenright) obtuvo la mayor precisión (91.75 %), indicando que cuando el modelo predice esta clase, generalmente está en lo correcto. La clase 2 mostró la precisión más baja (77.19 %).
- **Recall (Sensibilidad)**: la clase 3 alcanzó el recall más alto (91.81 %), demostrando que el modelo detecta exitosamente la mayoría de los casos de reina aceptada. La clase 0 presentó el recall más bajo (77.62 %), sugiriendo que algunos casos de colmenas funcionales fueron mal clasificados.
- **F1-Score**: representa el equilibrio entre precisión y recall. La clase 3 obtuvo el mejor F1-score (0.9162), mientras que la clase 1 presentó el menor valor (0.7884).

4.3. Análisis Temporal de Predicciones

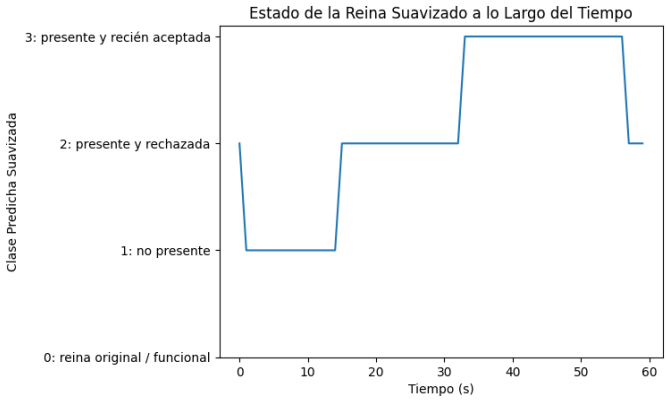
Las figuras 4.1 y 4.2 presentan los resultados de la aplicación del modelo en grabaciones de audio continuas, mostrando la evolución temporal de la confianza del clasificador y las predicciones suavizadas mediante ventana móvil de 60 segundos.

La figura 4.1a muestra alta variabilidad en la confianza instantánea del primer modelo evaluado, con valores oscilando entre 0.3 y 1.0, lo que sugiere inestabilidad en las predicciones a corto plazo. En contraste, la figura 4.2a demuestra que el segundo modelo mantiene confianzas consistentemente superiores al 95 %, con únicamente dos caídas puntuales, indicando mayor robustez y estabilidad temporal.

Las predicciones suavizadas (figuras 4.1b y 4.2b) revelan que el segundo modelo logra transiciones más definidas entre estados, esencial para aplicaciones de monitoreo en tiempo real donde se requiere identificar cambios en el estado de la reina con alta certeza.

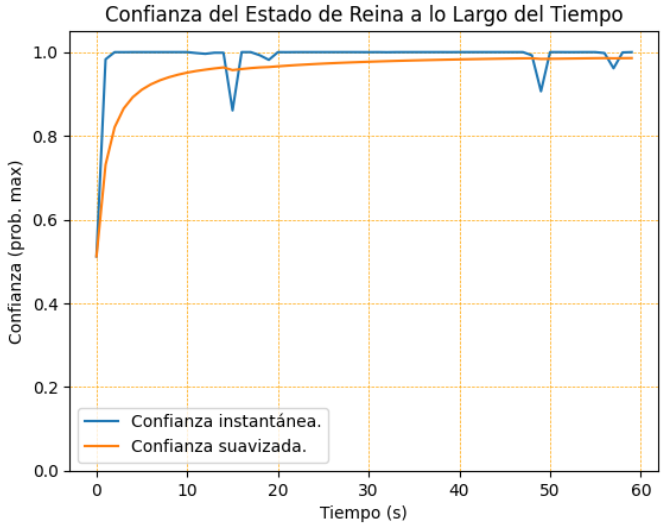


(a) Evolución temporal de la confianza del modelo según `queen status`.

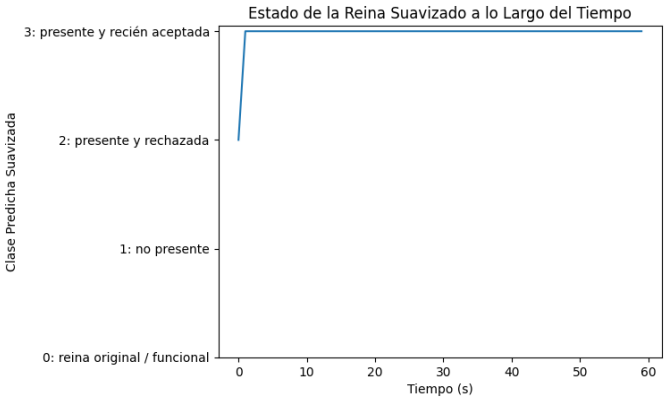


(b) Clase predicha suavizada a lo largo del tiempo según `queen status`.

Figura 4.1: Resultados del primer modelo de clasificación para el estado de la reina a lo largo del tiempo.



(a) Evolución temporal de la confianza del modelo según `queen status`.



(b) Clase predicha suavizada a lo largo del tiempo según `queen status`.

Figura 4.2: Resultados del segundo modelo de clasificación para el estado de la reina a lo largo del tiempo.

5. DISCUSIÓN

5.1. Interpretación de los Resultados

Los resultados presentados en las figuras 4.1 y 4.2 evidencian diferencias significativas en el desempeño de los dos modelos CNN implementados para la clasificación del estado de la reina en colmenas mediante análisis acústico basado en MFCCs.

5.1.1. Análisis del Primer Modelo (Figura 4.1)

El primer modelo presenta una alta variabilidad en la confianza instantánea, con valores que oscilan entre 0.3 y 1.0 a lo largo del tiempo, tal como se muestra en la figura 4.1a. Esta inestabilidad sugiere dificultades para mantener predicciones consistentes en ventanas temporales consecutivas. Esto se puede atribuir a varios factores:

- **Sensibilidad al ruido temporal:** la variabilidad observada indica que el modelo podría estar sobreajustándose a fluctuaciones de corto plazo en las características acústicas, en lugar de capturar patrones estructurales robustos y generalizables del comportamiento sonoro de la colmena.
- **Limitaciones en el modelado temporal:** aunque las CNN son capaces de capturar patrones locales en representaciones espectrales, la ausencia de mecanismos explícitos de memoria (por ejemplo, capas LSTM) limita la integración de información contextual entre ventanas consecutivas.
- **Desbalance de clases:** a pesar del uso de pesos de clase durante el entrenamiento, la inestabilidad sugiere posibles sesgos hacia clases mayoritarias o dificultades para discriminar patrones acústicos sutiles.

La curva de confianza suavizada (promedio móvil de 60 segundos) muestra una estabilización en el rango de 0.4–0.6, lo cual representa una mejora respecto a las predicciones instantáneas, aunque sigue siendo inferior a lo reportado en la literatura. En particular, Ruvinga et al. reportan una precisión de 99.31 % utilizando CNN sobre espectrogramas STFT [9].

La clase predicha suavizada (figura 4.1b) muestra transiciones frecuentes entre clases, lo que dificulta la identificación de períodos prolongados en donde se pueda determinar con precisiónla presencia de una reina, limitando así la utilidad práctica del modelo para monitoreo en tiempo real. Además, con este audio en particular; la predicción fue errónea. Esto que sugiere que el modelo no ha logrado capturar patrones acústicos relevantes para esta grabación específica, posiblemente debido a variaciones en las condiciones de grabación o características acústicas atípicas.

5.1.2. Análisis del Segundo Modelo (Figura 4.2)

El segundo modelo presenta una mejora sustancial en todos los indicadores de desempeño. La confianza instantánea se mantiene consistentemente por encima del 95 % durante la mayor parte de la secuencia temporal, con únicamente dos caídas puntuales expuestas en la figura 4.2a. Entonces, se pueden extraer las siguientes conclusiones:

- **Estabilidad temporal:** la alta confianza sostenida indica que el modelo ha aprendido representaciones más robustas y generalizables. Esta estabilidad es esencial para aplicaciones de monitoreo en tiempo real.

- **Transiciones bien definidas:** se observa una transición clara entre estados, lo cual coincide con una caída momentánea de confianza, consistente con un cambio en el régimen acústico.

- **Posibles eventos de incertidumbre:**

  - Eventos transitorios reales en la colmena.
  - Artefactos de grabación o ruido externo.
  - Limitaciones en la segmentación temporal.

  La rápida recuperación de la confianza sugiere que estos eventos son aislados y que el suavizado temporal es efectivo.

La clase predicha suavizada presentada en la figura 4.2b, muestra una transición clara entre clases, lo que sugiere que el modelo ha logrado capturar patrones acústicos relevantes específicamente para esta grabación, siendo acertivo en su predicción. Sin embargo, la presencia de caídas puntuales en la confianza indica que aún existen dificultades para mantener un desempeño óptimo en cuanto a todas las condiciones de grabación.

5.2. Comparaciones

5.2.1. Comparación entre Modelos

Comparando ambos modelos, se observa que el segundo modelo ha logrado superar las limitaciones del primero, alcanzando una mayor estabilidad y precisión en la clasificación del estado de la reina. El cambio se debe a modificaciones en la función de activación, pasando de ser ReLU a una función de activación más suave como lo es sigmoid, lo que ha permitido al modelo aprender representaciones más discriminativas y generalizables. Sin embargo, la presencia de eventos de incertidumbre en ambos modelos indica que aún existen desafíos relacionados con la variabilidad acústica y las condiciones de grabación, lo que resalta la necesidad de continuar explorando técnicas para mejorar la robustez y generalización de los modelos.

5.2.2. Comparación con el Estado del Arte

Los resultados del segundo modelo son comparables con los reportados por Ruvinga et al. [4, 9].

- **Representación de entrada:** a diferencia de los espectrogramas STFT utilizados en trabajos previos, el uso de MFCCs proporciona una representación más compacta y perceptualmente relevante.

- **Clasificación multiclase:** este trabajo aborda un problema más complejo con cuatro clases, en contraste con enfoques binarios.

- **Análisis temporal:** se incorpora evaluación de consistencia temporal, aspecto poco explorado en estudios previos.

5.3. Limitaciones y Desafíos Identificados

5.3.1. Generalización

Los datos provienen de condiciones controladas, por lo que la generalización a otros entornos requiere validación adicional. Factores como clima, raza de abejas y condiciones ambientales pueden influir en los patrones acústicos [11].

5.3.2. Desbalance de Clases

El dataset presenta desbalance al mostrar mayor cantidad de datos para una clase con respecto a las demás, lo cual puede afectar el desempeño. La cantidad de etiquetas por clase se muestra a continuación:

- `queenright/original queen` (0): 179.
- `not present` (1): 158.
- `present and rejected` (2): 259.
- `present and newly accepted` (3): 679.

5.3.3. Robustez al Ruido

El ruido ambiental puede degradar el desempeño del modelo [14]. Técnicas de denoising o modelos más robustos podrían ser necesarios para aplicaciones en campo.

5.3.4. Implicaciones Prácticas

Los resultados demuestran la viabilidad de sistemas de monitoreo acústico no invasivo.

- Reducción de inspecciones invasivas.
- Detección temprana de la ausencia de una reina.
- Escalabilidad mediante dispositivos IoT.
- Contribución a la sustentabilidad apícola [10, 11].

5.3.5. Direcciones Futuras

Se proponen las siguientes líneas de investigación:

- Validación con múltiples datasets.
- Modelos híbridos CNN-LSTM.
- Transfer learning.
- Detección de eventos acústicos específicos.
- Implementación en hardware embebido.
- Integración multimodal.

6. CONCLUSIONES Y RECOMENDACIONES

6.1. Conclusiones

La abjea reina es un elemento crucial para la supervivencia de la colonia, y su presencia o ausencia puede influir significativamente en el comportamiento de las abejas, siendo una posibilidad el extingir la existensia de la colmena.

Bibliografía

[1] Sofía de León Guedes, Estela Santos, Arnaldo Moreni et al. «Calidad nutricional del polen de colza (Brassica napus) y determinación de la flora apícola acompañante para las abejas melíferas». En: (2023).

[2] ML Winston. *Temperate and Tropical Honey Bees in The Biology of the Honey Bee. 214–224*. 1987.

[3] M. Beekman y F. L. W. Ratnieks. «Long-range foraging by the honey-bee, Apis mellifera L». En: *Functional Ecology* 14 (2000), págs. 490-496. DOI: 10.1046/j.1365-2435.2000.00443.x.

[4] Stenford Ruvinga et al. «Use of LSTM networks to identify "queenlessness" in honeybee hives from audio signals». En: *2021 17th International Conference on Intelligent Environments (IE)*. IEEE. 2021, págs. 1-4.

[5] Zhengwei Wang et al. «Encoding and decoding of the information in the honeybee waggle dance». En: *Behavioral Ecology and Sociobiology* 79.5 (2025), pág. 53.

[6] Brian R Johnson. «Division of labor in honeybees: form, function, and proximate mechanisms». En: *Behavioral ecology and sociobiology* 64 (2010), págs. 305-316.

[7] Hichem Maaref y Dominique Fourer. «Remote Beehive Health Analysis using Embedded System and Relevant Audio Features». En: ().

[8] Donald Howard et al. «Signal processing the acoustics of honeybees (apis mellifera) to identify the 'queenless' state in hives». En: *Proceedings of the Institute of Acoustics* 35.1 (2013), pág. 290.

[9] Stenford Ruvinga et al. «Identifying queenlessness in honeybee hives from audio signals using machine learning». En: *Electronics* 12.7 (2023), pág. 1627.

[10] Shaden AM Khalifa et al. «Overview of bee pollination and its economic value for crop production». En: *Insects* 12.8 (2021), pág. 688.

[11] Simon G Potts et al. «Safeguarding pollinators and their values to human well-being». En: *Nature* 540.7632 (2016), págs. 220-229.

[12] Sara Ferrari et al. «Monitoring of swarming sounds in bee hives for early detection of the swarming period». En: *Computers and electronics in agriculture* 64.1 (2008), págs. 72-77.

[13] Antonio Robles Guerrero et al. «Análisis de patrones acústicos para determinar estado de colonias de abejas mediante Regresión Logística Lasso y Descomposición en Valores Singulares». En: (2019).

[14] Rui Peng, Iman Ardekani y Hamid Sharifzadeh. «An acoustic signal processing system for identification of queen-less beehives». En: *2020 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*. IEEE. 2020, págs. 57-63.

[15] Stanley Smith Stevens, John Volkmann y Edwin Broomell Newman. «A scale for the measurement of the psychological magnitude pitch». En: *The journal of the acoustical society of america* 8.3 (1937), págs. 185-190.

[16] H Shimodaira y S Rennals. *Speech Signal Analysis*. https://www.inf.ed.ac.uk/teaching/courses/asr/2012-13/asr02-signal-4up.pdf. 2013.

[17] Fang Zheng, Guoliang Zhang y Zhanjiang Song. «Comparison of different implementations of MFCC». En: *Journal of Computer Science and Technology* 16.6 (2001), págs. 582-589.

[18] Sepp Hochreiter y Jürgen Schmidhuber. «Long short-term memory». En: *Neural computation* 9.8 (1997), págs. 1735-1780.

[19] Anna Yang. *Smart Bee Colony Monitor: Clips of Beehive Sounds*. https://www.kaggle.com/datasets/annajyang/beehive-sounds. [Online; accessed 29-April-2025]. 2023.

APÉNDICE A

NOTA: los acentos fueron removidos en la descrtipción de los programas por problemas de compatibilidad con la librería 'listing' en LaTeX.

Programa # 1

```
import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
from pathlib import Path

def peak_normalize(x, eps=1e-9):
    '''
    Normalizacion por pico (peak normalization).
    Se aplica despues de trim y antes de segmentar (a nivel de archivo completo)
    .
    - Args:
        x: senal de audio (1D numpy array)
        eps: epsilon para evitar division por cero
    - Returns:
        Audio normalizado con pico = 1.0
    '''
    return x / (np.max(np.abs(x)) + eps)

def segment_signal(x, sr, seg_sec, hop_sec):
    '''
    Segmenta la senal de audio en ventanas con overlap.
    Si un segmento es mas corto que seg_sec, se hace padding con reflect.
    - Args:
        x: senal de audio (1D numpy array)
        sr: sample rate
        seg_sec: duracion del segmento en segundos
        hop_sec: hop entre segmentos en segundos
    - Returns:
        Lista de segmentos de audio
    '''
    seg_len = int(seg_sec * sr)
    hop_len = int(hop_sec * sr)
```

```

    # Si el audio es mas corto que seg_len, hacer padding
    if len(x) < seg_len:
        x = np.pad(x, (0, seg_len - len(x)), mode='reflect')

    segments = []
    for start in range(0, max(1, len(x) - seg_len + 1), hop_len):
        seg = x[start:start + seg_len]
        if len(seg) < seg_len:
            seg = np.pad(seg, (0, seg_len - len(seg)), mode='reflect')
        segments.append(seg)
    return segments

def extract_mfcc_features(audio_segment):
    '''
    Extrae MFCCs siguiendo el pipeline de produccion:
    1. Extrae N_MFCC_RAW (33) MFCCs
    2. Elimina C0 (primer coeficiente) - queda (32, T)
    3. Calcula delta y delta-delta sobre los 32 coeficientes
    4. Apila - (3, 32, T) si ADD_DELTAS=True
    - Args:
        audio_segment: senal de audio (1D numpy array)
    - Returns:
        features: numpy array con shape (channels, N_MFCC_FINAL, T_frames)
                  channels = 3 si ADD_DELTAS=True, else 1
    '''
    # Extraer MFCCs con C0 incluido
    mfcc = librosa.feature.mfcc(
        y=audio_segment,
        sr=SR,
        n_mfcc=N_MFCC_RAW,      # 33 coeficientes
        n_fft=N_FFT,
        hop_length=HOP_LEN,
        fmin=FMIN,
        fmax=FMAX
    )
    # Eliminar C0 (primer coeficiente) - shape: (32, T)
    mfcc = mfcc[1:, :]

    if ADD_DELTAS:
        # Calcular deltas sobre los 32 coeficientes (sin C0)
        delta = librosa.feature.delta(mfcc)
        delta2 = librosa.feature.delta(mfcc, order=2)
```

```
        # Apilar: (3, 32, T)
        features = np.stack([mfcc, delta, delta2], axis=0)
    else:
        # Solo MFCCs: (1, 32, T)
        features = mfcc[np.newaxis, :, :]
    return features.astype(np.float32)

def preprocess_audio_file(audio_path):
    '''
    Pipeline completo de preprocesamiento consistente con produccion:
  - Args:
        audio_path: ruta al archivo de audio
  - Returns:
        segments_features: lista de arrays con shape (3, 32, T_frames)
        metadata: diccionario con informacion del procesamiento
    '''
    audio_path = Path(audio_path)

    # 1. Load + resample
    audio_raw, _ = librosa.load(str(audio_path), sr=SR, mono=True)

    # 2. Trim
    audio_trimmed, _ = librosa.effects.trim(audio_raw, top_db=TRIM_DB)

    # 3. Peak normalization (a nivel de archivo)
    audio_normalized = peak_normalize(audio_trimmed)

    # 4. Segmentar
    segments = segment_signal(audio_normalized, SR, SEG_SEC, HOP_SEC)

    # 5. Extraer features por segmento
    segments_features = []
    for i, seg in enumerate(segments):
        feat = extract_mfcc_features(seg)
        segments_features.append(feat)

    # Metadata
    metadata = {
        'filename': audio_path.name,
        'duration_original': len(audio_raw) / SR,
        'duration_trimmed': len(audio_trimmed) / SR,
        'num_segments': len(segments),
        'feature_shape': segments_features[0].shape if segments_features else
```

```
    None
    }
    return segments_features, metadata

# PARAMETROS DE PREPROCESAMIENTO

# Audio Configuration
SR = 16000                # Sample rate (resample todo a 16kHz)
TRIM_DB = 30              # Top dB para trim (eliminar silencios)

# Segmentacion
SEG_SEC = 2.0             # Duracion de cada segmento: 2 segundos
HOP_SEC = 1.0             # Hop entre segmentos: 1 segundo (50% overlap)

# MFCC FE
# Ventana: 25ms, Hop: 10ms (estandar en ASR)
N_FFT = int(0.025 * SR)       # 25ms - 400 muestras
HOP_LEN = int(0.010 * SR)     # 10ms - 160 muestras

# Frecuencias
FMIN = 20                     # Frecuencia minima (Hz)
FMAX = SR // 2                # Frecuencia maxima (Nyquist): 8000 Hz

# Coeficientes MFCC
N_MFCC_RAW = 33               # Extraemos 33 MFCCs inicialmente
N_MFCC_FINAL = 32             # Eliminamos C0 - 32 coeficientes finales
ADD_DELTAS = True             # Incluir delta y delta-delta

AUDIO_PATH = r'<<archivo de audio>>'
features, metadata = preprocess_audio_file(AUDIO_PATH, verbose=True)

# Directorio con audios
DATA_DIR = Path(r'<<directorio con archivos de audio>>')

# Buscar los primeros 5 archivos WAV
audio_files = list(DATA_DIR.rglob('*.wav'))[:5]

if len(audio_files) == 0:
    print('No se encontraron archivos .wav en el directorio.')
else:
    print(f'\n{'='*70}')
    print(f'TEST BATCH: Procesando {len(audio_files)} archivos')
    print(f'{'='*70}\n')
```

```
    results = []
    for audio_path in audio_files:
        try:
            features, metadata = preprocess_audio_file(audio_path, verbose=False)
            results.append({
                'filename': metadata['filename'],
                'num_segments': metadata['num_segments'],
                'shape': metadata['feature_shape']
            })
            print(f'{metadata['filename']:<30} - {metadata['num_segments']:>3} segmentos')
        except Exception as e:
            print(f'{audio_path.name:<30} - ERROR: {e}')

    print(f'\n{'='*70}')
    print(f'Procesamiento batch completado: {len(results)}/{len(audio_files)} exitosos')
    print(f'{'='*70}')

# Cargar un segmento del dataset de entrenamiento
NPY_DIR = Path(r'<<directorio con archivo *.npy>>')

try:
    X_train = np.load(NPY_DIR / 'X_train.npy', mmap_mode='r')
    print(f'\n{'='*70}')
    print('VALIDACION DE CONSISTENCIA')
    print(f'{'='*70}')
    print(f'Shape del dataset de entrenamiento: {X_train.shape}')
    print(f'   Esperado: (N_samples, 3, 32, T_frames)')
    print(f'\nShape de nuestro test:')
    if 'features' in locals() and len(features) > 0:
        print(f'   {features[0].shape}')
        print(f'\n Las dimensiones coinciden.' if features[0].shape[0] == 3 and features[0].shape[1] == 32 else 'Las dimensiones NO coinciden')

    # Estadisticas comparativas
    print(f'\n Estadisticas de X_train (primer segmento):')
    sample = X_train[0].astype(np.float32)
    print(f'   Min:  {np.min(sample):.4f}')
    print(f'   Max:  {np.max(sample):.4f}')
    print(f'   Mean: {np.mean(sample):.4f}')
```

```
    print(f'   Std:  {np.std(sample):.4f}')
    if 'features' in locals() and len(features) > 0:
        print(f'\n Estadisticas de nuestro test (primer segmento):')
        print(f'   Min:  {np.min(features[0]):.4f}')
        print(f'   Max:  {np.max(features[0]):.4f}')
        print(f'   Mean: {np.mean(features[0]):.4f}')
        print(f'   Std:  {np.std(features[0]):.4f}')
    print(f'{'='*70}\n')

except FileNotFoundError:
    print('No se encontraron los archivos .npy de entrenamiento.')
```

Listing A.1: Prueba de extracción de características acústicas (MFCC) y validación de la representación numérica del audio.o

Programa # 2

```python
# CELDA 1:
# ----------------------------------------------------------------------
from pathlib import Path
import pandas as pd
import numpy as np
import librosa
from sklearn.model_selection import train_test_split
import json
# ----------------------------------------------------------------------

# CELDA 2:
# ----------------------------------------------------------------------
def peak_normalize(x, eps=1e-9):
    return x / (np.max(np.abs(x)) + eps)

def load_and_clean(path):
    '''
    Preprocesamiento a nivel de archivo (CONTRATO DE PREPROCESAMIENTO):
    1. Load + resample a SR=16000
    2. Trim con TRIM_DB=30 (elimina silencios al inicio/fin)
    3. Peak normalization sobre el audio TRIMMEADO (a nivel de archivo completo)
    '''
    x, _ = librosa.load(str(path), sr=SR, mono=True)
    x, _ = librosa.effects.trim(x, top_db=TRIM_DB)
    x = peak_normalize(x)
    return x

def segment_signal(x, sr, seg_sec, hop_sec):
    seg_len = int(seg_sec * sr)
    hop_len = int(hop_sec * sr)
    if len(x) < seg_len:
        x = np.pad(x, (0, seg_len - len(x)), mode='reflect')
    segments = []
    for start in range(0, max(1, len(x)-seg_len+1), hop_len):
        seg = x[start:start+seg_len]
        if len(seg) < seg_len:
            seg = np.pad(seg, (0, seg_len - len(seg)), mode='reflect')
        segments.append(seg)
    return segments

def mfcc_features(seg):
    '''
```

```
    Extrae MFCCs + deltas con C0 eliminado.
    Contrato de preprocesamiento:
    1. Se recibe 'seg' ya trimmeado, normalizado y segmentado
    2. Extraemos 33 MFCCs (n_mfcc=33)
    3. Eliminamos C0 -> queda (32, T)
    4. Calculamos Delta y Delta-Delta sobre los 32 coefs restantes
    5. Apilamos -> (3, 32, T)
    '''
    # Extraer 33 MFCCs para luego eliminar C0
    mfcc = librosa.feature.mfcc(
        y=seg, sr=SR, n_mfcc=33, n_fft=N_FFT, hop_length=HOP_LEN,
        fmin=FMIN, fmax=FMAX
    )
    # Eliminar C0 (primer coeficiente) -> ahora (32, T)
    mfcc = mfcc[1:, :]

    if ADD_DELTAS:
        d1 = librosa.feature.delta(mfcc)
        d2 = librosa.feature.delta(mfcc, order=2)
        feat = np.stack([mfcc, d1, d2], axis=0)  # (3, 32, T)
    else:
        feat = mfcc[np.newaxis, :, :]            # (1, 32, T)
    return feat.astype(np.float32)
# ---------------------------------------------------------------------------

# CELDA 3:
# ---------------------------------------------------------------------------
DATA_DIR = Path(r'<<directorio con archivos de audio>>')
CSV_PATH = Path(r'<<archivo *.csv con etiquetas>>')

ID_COL = 'file name'            # <-- columna de nombres
TARGET_COL = 'queen status'     # <-- la etiqueta que se utilizara
VALID_CLASSES = {0, 1, 2, 3}

# 1) Construir diccionario: nombre_archivo.wav -> ruta_completa
wav_paths = sorted(list(DATA_DIR.rglob('*.wav'))) + sorted(list(DATA_DIR.rglob('
    *.WAV')))
name_to_path = {p.name: str(p) for p in wav_paths}  # str() para que sea
    serializable y facil

df = pd.read_csv(CSV_PATH)

assert ID_COL in df.columns, f'Falta ID_COL={ID_COL} en el CSV'
```

```python
assert TARGET_COL in df.columns, f'Falta TARGET_COL={TARGET_COL} en el CSV'

# 2) Normaliza nombres (solo basename)
df['_basename'] = df[ID_COL].astype(str).map(lambda s: Path(s).name)

# 3) Convertir queen status a numerico y filtrar clases validas
df['_y'] = pd.to_numeric(df[TARGET_COL], errors='coerce')
labeled = df[df['_y'].isin([0,1,2,3])].copy()
labeled['y'] = labeled['_y'].astype(np.int64)

# 4) Crear columna 'path' (ruta completa al wav)
labeled['path'] = labeled['_basename'].map(name_to_path)

# 5) Verificar que no falte ninguno
missing = labeled['path'].isna().sum()
print('Etiquetados:', len(labeled), '| faltantes sin path:', int(missing))
assert missing == 0, 'Hay archivos etiquetados que no se encontraron en la carpeta.'
print('Conteo por clase:\n', labeled['y'].value_counts().sort_index())

# Audio / MFCC params
SR = 16000
TRIM_DB = 30
SEG_SEC = 2.0
HOP_SEC = 1.0
N_MFCC = 32
N_FFT  = int(0.025 * SR)
HOP_LEN= int(0.010 * SR)
FMIN, FMAX = 20, SR//2
ADD_DELTAS = True
RANDOM_SEED = 123
TEST_SIZE = 0.15
VAL_SIZE  = 0.15

# Direccion para guardar los archivos .npy con las caracteristicas MFCC
OUT_DIR = Path(r'<<directorio con archivos de audio>>')
OUT_DIR.mkdir(parents=True, exist_ok=True)
# ---------------------------------------------------------------------

# CELDA 4:
# ---------------------------------------------------------------------
paths = labeled['path'].values
ys    = labeled['y'].values
```

```python

# paths, ys ya definidos (por archivo, no por segmento)
paths_train, paths_tmp, y_train, y_tmp = train_test_split(
    paths, ys, test_size=(TEST_SIZE + VAL_SIZE), random_state=RANDOM_SEED, stratify=ys
)

# aqui tmp es (val+test); lo partimos a la mitad si VAL_SIZE==TEST_SIZE
rel_test = TEST_SIZE / (TEST_SIZE + VAL_SIZE)

paths_val, paths_test, y_val, y_test = train_test_split(
    paths_tmp, y_tmp, test_size=rel_test, random_state=RANDOM_SEED, stratify=y_tmp
)

print('Train/Val/Test:', len(paths_train), '/', len(paths_val), '/', len(paths_test))
# ---------------------------------------------------------------------------

# CELDA 5:
# ---------------------------------------------------------------------------
def process_labeled_file_list(file_list, label_list):
    feats = []
    labs = []
    file_index = []  # trazabilidad: (archivo, segmento)
    for p, y in zip(file_list, label_list):
        x = load_and_clean(p)
        segs = segment_signal(x, SR, SEG_SEC, HOP_SEC)
        for k, seg in enumerate(segs):
            feats.append(mfcc_features(seg))
            labs.append(y)
            file_index.append({'file': str(p), 'segment': int(k)})
    X = np.stack(feats, axis=0)  # (Nseg, C, n_mfcc, T)
    y = np.array(labs, dtype=np.int64)
    return X, y, file_index

print('Extrayendo TRAIN...')
X_train, y_train_seg, idx_train = process_labeled_file_list(paths_train, y_train)
print('Extrayendo VAL...')
X_val, y_val_seg, idx_val = process_labeled_file_list(paths_val, y_val)
print('Extrayendo TEST...')
X_test, y_test_seg, idx_test = process_labeled_file_list(paths_test, y_test)
```

```python

print('Shapes:')
print('X_train:', X_train.shape, 'y:', y_train_seg.shape)
print('X_val:', X_val.shape, 'y:', y_val_seg.shape)
print('X_test:', X_test.shape, 'y:', y_test_seg.shape)
# ---------------------------------------------------------------------------

# CELDA 6:
# ---------------------------------------------------------------------------
X_train = X_train.astype(np.float16)
X_val   = X_val.astype(np.float16)
X_test  = X_test.astype(np.float16)

np.save(OUT_DIR / 'X_train.npy', X_train)
np.save(OUT_DIR / 'y_train.npy', y_train_seg)
np.save(OUT_DIR / 'X_val.npy',   X_val)
np.save(OUT_DIR / 'y_val.npy',   y_val_seg)
np.save(OUT_DIR / 'X_test.npy',  X_test)
np.save(OUT_DIR / 'y_test.npy',  y_test_seg)
# ---------------------------------------------------------------------------
```

Listing A.2: Generando archivos *.npy (En Jupyter Notebooks *.ipynb).

Programa # 3

```python
# CELDA 1:
# ----------------------------------------------------------------------
from pathlib import Path
import os, random
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix

SEED = 123

os.environ['PYTHONHASHSEED'] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

BASE = Path(r'<<Directorio con archivos *.npy>>')

X_train = np.load(BASE / 'X_train.npy', mmap_mode='r')
y_train = np.load(BASE / 'y_train.npy')
X_val   = np.load(BASE / 'X_val.npy',   mmap_mode='r')
y_val   = np.load(BASE / 'y_val.npy')
X_test  = np.load(BASE / 'X_test.npy',  mmap_mode='r')
y_test  = np.load(BASE / 'y_test.npy')

print('X_train:', X_train.shape, X_train.dtype)
print('y_train:', y_train.shape, y_train.dtype)
print('X_val:  ', X_val.shape, X_val.dtype)
print('X_test: ', X_test.shape, X_test.dtype)

# Chequeo de clases
u, c = np.unique(y_train, return_counts=True)
print('Distribucion train:', dict(zip(u, c)))
# ----------------------------------------------------------------------

# CELDA 2:
# ----------------------------------------------------------------------
BATCH = 64

def make_ds(X, y, training=False):
    ds = tf.data.Dataset.from_tensor_slices((X, y))
    if training:
```

```
        ds = ds.shuffle(20000, reshuffle_each_iteration=True)
    ds = ds.batch(BATCH)
    ds = ds.map(lambda a,b: (tf.cast(a, tf.float32), b), num_parallel_calls=tf.
   data.AUTOTUNE)
    ds = ds.prefetch(tf.data.AUTOTUNE)
    return ds

train_ds = make_ds(X_train, y_train, training=True)
val_ds   = make_ds(X_val, y_val, training=False)
test_ds  = make_ds(X_test, y_test, training=False)
# ---------------------------------------------------------------------------

# CELDA 3:
# ---------------------------------------------------------------------------
num_classes = 4
counts = np.bincount(y_train, minlength=num_classes)
total = counts.sum()

# Peso inverso a frecuencia (simple y efectivo)
class_weight = {i: float(total / (num_classes * counts[i])) for i in range(
   num_classes) if counts[i] > 0}
print('counts:', counts)
print('class_weight:', class_weight)
# ---------------------------------------------------------------------------

# CELDA 4:
# ---------------------------------------------------------------------------
num_classes = 4
input_shape = (3, 32, 201)
act_fn = 'sigmoid'

model = models.Sequential([
    layers.Input(shape=input_shape),
    layers.Conv2D(16, (3,3), padding='same', activation=act_fn, data_format='
   channels_first'),
    layers.BatchNormalization(axis=1),
    layers.MaxPool2D((2,2), data_format='channels_first'),

    layers.Conv2D(32, (3,3), padding='same', activation=act_fn, data_format='
   channels_first'),
    layers.BatchNormalization(axis=1),
    layers.MaxPool2D((2,2), data_format='channels_first'),

```

```
    layers.Conv2D(64, (3,3), padding='same', activation=act_fn, data_format='channels_first'),
    layers.BatchNormalization(axis=1),
    layers.MaxPool2D((2,2), data_format='channels_first'),

    layers.Conv2D(128, (3,3), padding='same', activation=act_fn, data_format='channels_first'),
    layers.BatchNormalization(axis=1),
    layers.GlobalAveragePooling2D(data_format='channels_first'),

    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(num_classes, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
# ---------------------------------------------------------------------------

# CELDA 5:
# ---------------------------------------------------------------------------
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_accuracy', mode='max',
        patience=10, restore_best_weights=True
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_accuracy', mode='max',
        patience=3, factor=0.5
    ),
    tf.keras.callbacks.ModelCheckpoint(
        filepath=str(BASE / 'cnn_mfcc_queenRec1.h5'),
        monitor='val_accuracy', mode='max',
        save_best_only=True
    )
]

history = model.fit(
```

```
    train_ds,
    validation_data=val_ds,
    epochs=40,
    class_weight=class_weight,
    callbacks=callbacks
)
# ---------------------------------------------------------------------------

# CELDA 6:
# ---------------------------------------------------------------------------
probs = model.predict(test_ds)
y_pred = np.argmax(probs, axis=1)

print('Confusion matrix:\n', confusion_matrix(y_test, y_pred))
print('\nClassification report:\n', classification_report(y_test, y_pred, digits
    =4))
# ---------------------------------------------------------------------------
```

Listing A.3: Extracción de características.

Programa # 3

```
# CELDA 1:
# ---------------------------------------------------------------------------
def peak_normalize(x, eps=1e-9):
    m = np.max(np.abs(x))
    return x if m < eps else (x / (m + eps))

def extract_mfcc_3ch(window_audio: np.ndarray) -> np.ndarray:
    '''
    Input: 1D float array of length WINDOW_SEC*SR
    Output: (3, 32, 201) float32  [channels_first]
    '''
    # Ajustar normalizacion de amplitud al estilo de entrenamiento (aprox.)
    w = peak_normalize(window_audio.astype(np.float32))

    mfcc = librosa.feature.mfcc(
        y=w, sr=SR,
        n_mfcc=N_MFCC,
        n_fft=N_FFT,
        hop_length=HOP_LEN,
        fmin=FMIN, fmax=FMAX
    )  # (32, T)

    if ADD_DELTAS:
        d1 = librosa.feature.delta(mfcc)
        d2 = librosa.feature.delta(mfcc, order=2)
        feat = np.stack([mfcc, d1, d2], axis=0)  # (3, 32, T)
    else:
        feat = mfcc[np.newaxis, :, :]            # (1, 32, T)

    return feat.astype(np.float32)
# ---------------------------------------------------------------------------

# CELDA 2:
# ---------------------------------------------------------------------------
import time
import numpy as np
import librosa
import matplotlib.pyplot as plt
from pathlib import Path
from collections import deque
import tensorflow as tf

```

```
# ----------------------------
# Direcciones
# ----------------------------
MODEL_PATH = Path(r'<<Modelo (red neuronal>>')
WAV_PATH   = Path(r'<<Archivo *.wav')

assert MODEL_PATH.exists(), f'Model not found: {MODEL_PATH}'
assert WAV_PATH.exists(),   f'WAV not found: {WAV_PATH}'

# ----------------------------
# Parametros
# ----------------------------
SR = 16000
WINDOW_SEC = 2.0
HOP_SEC = 1.0
SMOOTH_SEC = 60.0

# Parametros MFCC
N_MFCC = 32
N_FFT = int(0.025 * SR)      # 25 ms -> 400
HOP_LEN = int(0.010 * SR)    # 10 ms -> 160
FMIN, FMAX = 20, SR // 2
ADD_DELTAS = True

SIMULATE_REALTIME = True   # True = sleep 1s per hop, False = run fast

# Mapeo de etiquetas
CLASS_NAME = {
    0: '0: reina original / funcional',
    1: '1: no presente',
    2: '2: presente y rechazada',
    3: '3: presente y recien aceptada'
}

print('Config OK.')
# ---------------------------------------------------------------------------

# CELDA 3:
# ---------------------------------------------------------------------------
model = tf.keras.models.load_model(MODEL_PATH)
print('Loaded model:', MODEL_PATH)
# ---------------------------------------------------------------------------

```

```
# CELDA 4:
# ---------------------------------------------------------------------------
# Cargar como audio monofonico a la frecuencia de muestreo (SR)
audio, sr = librosa.load(str(WAV_PATH), sr=SR, mono=True)

print('Loaded WAV:', WAV_PATH.name)
print('sr:', sr, 'samples:', len(audio), 'duration (s):', len(audio)/sr)
# ---------------------------------------------------------------------------

# CELDA 5:
# ---------------------------------------------------------------------------
win_len = int(WINDOW_SEC * SR)
hop_len = int(HOP_SEC * SR)
smooth_N = int(SMOOTH_SEC / HOP_SEC)

# Buffer circular para el audio
buf = np.zeros(win_len, dtype=np.float32)
write_idx = 0

# Suavizado mediante suma acumulada (running sum)
prob_hist = deque(maxlen=smooth_N)
prob_sum = np.zeros((4,), dtype=np.float64)  # corriendo suma de probabilidades

times, conf_inst, conf_smooth, pred_inst, pred_smooth = [], [], [], [], []

t0 = time.time()
num_hops = int(np.ceil(len(audio) / hop_len))

print(f'Running {num_hops} hops | window={WINDOW_SEC}s hop={HOP_SEC}s smooth={
    SMOOTH_SEC}s ({smooth_N} preds)')

pos = 0
for step in range(num_hops):
    # Tomar el siguiente fragmento (chunk) del salto (hop)
    chunk = audio[pos:pos+hop_len]
    pos += hop_len

    # Rellenar (padding) el ultimo fragmento si es necesario
    if len(chunk) < hop_len:
        chunk = np.pad(chunk, (0, hop_len - len(chunk)), mode='constant')
    chunk = chunk.astype(np.float32, copy=False)

    # escribir en el buffer circular
```

```
    end = write_idx + hop_len
    if end <= win_len:
        buf[write_idx:end] = chunk
    else:
        n1 = win_len - write_idx
        buf[write_idx:] = chunk[:n1]
        buf[:end - win_len] = chunk[n1:]
    write_idx = (write_idx + hop_len) % win_len

    # leer los ultimos 2s en el orden correcto (evitar exceso de roll/copy)
    if write_idx == 0:
        window = buf
    else:
        window = np.concatenate([buf[write_idx:], buf[:write_idx]])


    probs = model.predict(x, verbose=0)[0]       # (4,)
    y_hat = int(np.argmax(probs))
    c_hat = float(np.max(probs))

    # actualizar el suavizado de la media movil
    if len(prob_hist) == prob_hist.maxlen:
        prob_sum -= prob_hist[0]
    prob_hist.append(probs)
    prob_sum += probs
    probs_s = (prob_sum / len(prob_hist)).astype(np.float32)

    y_s = int(np.argmax(probs_s))
    c_s = float(np.max(probs_s))

    # STATE.update({'t': float(sec), 'class': int(y_s), 'confidence': float(c_s)
    })

    sec = step * HOP_SEC
    times.append(sec)
    conf_inst.append(c_hat)
    conf_smooth.append(c_s)
    pred_inst.append(y_hat)
    pred_smooth.append(y_s)

    if step % 5 == 0:  # imprimir cada 5 segundos para reducir spam/sobrecarga
        print(f't={sec:6.1f}s - inst={CLASS_NAME[y_hat]:28s} conf={c_hat:0.3f} -
    smooth={CLASS_NAME[y_s]:28s} conf={c_s:0.3f}')
```

```
    if SIMULATE_REALTIME:
        elapsed = time.time() - t0
        target = (step + 1) * HOP_SEC
        sleep_time = target - elapsed
        if sleep_time > 0:
            time.sleep(sleep_time)

print('Done.')
# ----------------------------------------------------------------------
```

Listing A.4: Extracción de características.